

Breaking Verifiable Delay Functions in the Random Oracle Model

Ziyi Guan

ziyi.guan@epfl.ch
EPFL

Artur Riazanov

artur.riazanov@epfl.ch
EPFL

Weiqiang Yuan

weiqiang.yuan@epfl.ch
EPFL

July 12, 2026

Abstract

This work resolves the open problem of whether verifiable delay functions (VDFs) can be constructed in the random oracle model. A VDF is a cryptographic primitive that requires a long time to compute (even with parallelization), but produces a unique output that is efficiently and publicly verifiable.

We prove that VDFs do not exist in the random oracle model. This also rules out black-box constructions of VDFs from other cryptographic primitives, such as one-way functions, one-way permutations and collision-resistant hash functions.

Prior to our work, Mahmoody, Smith and Wu (ICALP 2020) prove that *perfectly unique* VDFs (a much stronger form of VDFs) do not exist in the random oracle model; on the other hand, Ephraim, Freitag, Komargodski, and Pass (Eurocrypt 2020) construct VDFs in the random oracle model assuming the hardness of repeated squaring. Our result is optimal – we bridge the current gap between previously known impossibility results and existing constructions.

We initiate the study of *proof of work functions*, a new cryptographic primitive that shares similarities with both VDFs and proof of works. We show that a stronger form of it does not exist in the random oracle model, leaving open the fascinating possibility of a random-oracle-based construction.

Keywords: verifiable delay functions; proof of work functions; random oracle model; query complexity

Contents

1	Introduction	1
1.1	Our results	2
1.2	Related works	3
2	Techniques	5
2.1	From VDFs to search problems	5
2.2	Warm-up: VDFs with perfect uniqueness in the ROM	6
2.3	Statistically unique PoWFs in the ROM	7
2.4	VDFs with computational uniqueness in the ROM	8
3	Preliminaries	11
3.1	Verifiable delay functions in the random oracle model	11
3.2	Proof of work functions in the random oracle model	12
3.3	Search problems	13
3.4	VDFs to search problems	14
4	PoWFs with statistical uniqueness	16
4.1	Proof of Theorem 4.2	16
4.2	Finding the best leaf: proof of Theorem 4.5	17
4.3	Proof of Theorem 4.6	19
5	VDFs with computational uniqueness	21
5.1	The sequentiality breaker	22
5.2	The uniqueness breaker	23
5.3	Computational efficiency of the breakers	26
	Acknowledgments	28
A	Tightness of Theorem 5.1	28

1 Introduction

A verifiable delay function (VDF) [BBBF18] is a cryptographic primitive that requires a long *sequential* time to compute, while the output is efficiently verifiable. More specifically, a VDF is defined by two algorithms: Eval and Verify. On input x , Eval computes an output y and a proof π in time t_{Eval} , and Verify decides whether to accept (y, π) in time t_{Verify} , where $t_{\text{Verify}} \ll t_{\text{Eval}}$. The main security requirements for VDFs are *completeness*, *computational uniqueness* and *sequentiality*. Completeness says that the solution output by Eval is accepted by Verify with high probability. Computational uniqueness says that given an input x , no adversary running in time $\text{poly}(t_{\text{Eval}})$ can find a $y' \neq \text{Eval}(x)$ and a proof π' such that (y', π') convinces the verifier. Sequentiality says that no adversary running in time smaller than t_{Eval} (with parallel processors) can compute $y = \text{Eval}(x)$.

VDFs are useful in scenarios where a delay in the computation is needed to ensure that certain operations cannot be performed too quickly. It has applications in areas such as auction protocols, proof-of-work systems, cryptographic timestamping, secure multiparty computation, and building randomness beacons ([BBBF18; BBF18; Pie19; Wes19; FMPS19; EFKP20; Sta20; HHKK23]).

Another line of work using VDFs as building blocks is proving hardness of TFNP classes. Establishing the hardness of the TFNP class PPAD [Pap94], in which finding the Nash equilibrium of a non-cooperative game is the complete problem, is a long-standing open question. [BPR15; HY17; LV20; Bit+22] discuss the similarities between constructions of hard instances in PPAD and constructions of VDFs.

A natural question to study is whether black-box constructions of VDFs are possible from unstructured primitives, like hash functions or other symmetric primitives. The starting point would be to consider constructions in the random oracle model (ROM). [EFKP20] constructs VDFs in the ROM based on hardness of repeated squaring [RSW96]. [DGMV20] shows that *tight VDFs*, a variant whose evaluation time is very close to the sequentiality requirement, do not exist in the ROM. However, it is unclear whether the impossibility on tight VDFs extend to general VDFs. [MSW20] proves that VDFs satisfying *perfect uniqueness* (a strengthening of the computational uniqueness saying that no adversary can find a different solution) cannot be constructed in the ROM; they raise whether it is possible to rule out VDFs with computational uniqueness in the ROM as an open question.

We resolve the open question in [MSW20] and close the gap between existing constructions and known lower bounds by showing that:

Verifiable delay functions do not exist in the random oracle model.

In particular, we rule out black-box constructions of VDFs from a number of cryptographic primitives such as one-way functions, one-way permutations, and collision-resistant hash functions.

Proof of work functions. The above result indicates that sequentiality and uniqueness cannot coexist in the ROM. On the other hand, it is known that sequentiality alone does not cause problems — proof of sequential works (PoSWs) [DLM19] exist in the ROM. What about uniqueness? We study this question by proposing *proof of work functions (PoWFs)*, that can be viewed as either VDFs without sequentiality or proof of works (PoWs) [DN93; JJ99; Bac02] with computational uniqueness. We ask:

Can proof of work functions be constructed in the random oracle model?

PoWFs provide a better interface for some applications of PoWs. For example, in systems where resources are allocated based on PoWs [DN93; Bac02; JB99; LMCB12; GKL15], participants solve cryptographic puzzles as proof for their computational efforts to gain access to resources. However, standard PoW mechanisms do not prevent a user from finding multiple valid solutions after making the required effort once, thereby allowing them to exploit the system by submitting each valid solution separately. PoWFs address this risk by ensuring that only one valid solution can be found for each puzzle.

Moreover, the parallelizable nature makes PoWFs suitable for scenarios where VDFs are typically used, but delay is not necessary. PoWFs can efficiently provide unique outputs for applications like randomness beacons, which is useful in decentralized lotteries, random leader elections, etc., offering a compelling alternative when delay is not a primary concern.

1.1 Our results

Our first result shows that VDFs cannot simultaneously satisfy sequentiality and computational uniqueness in the ROM. This implies that VDFs cannot be constructed in the ROM. We then prove that a “strong” form of PoWFs also cannot exist in the ROM, which naturally raises the intriguing question of whether PoWFs might be realizable in the ROM.

Verifiable delay functions. Consider $\text{VDF} = (\text{Eval}, \text{Verify})$ in the ROM. We focus on the the number of queries made by Eval and Verify to the random oracle instead of their running time. Specifically, on input x , Eval computes an output y and a proof π with query complexity at most q_{Eval} , and Verify decides whether to accept (y, π) with query complexity at most q_{Verify} , where $q_{\text{Verify}} \ll q_{\text{Eval}}$. The uniqueness and sequentiality requirements are adapted accordingly.

Theorem 1 (Informal). *Suppose VDF is a VDF in the random oracle model that satisfies completeness and computational uniqueness. Then, there exists an adversary Adv that breaks the sequentiality of VDF by making at most $O(q_{\text{Verify}} \cdot q_{\text{Eval}})$ queries in $O(q_{\text{Verify}})$ rounds.*

We emphasize that our adversary Adv is *optimal* in the ROM – [EFKP20] constructs a VDF that satisfies both computational uniqueness and sequentiality in the ROM assuming the hardness of repeated squaring (the RSW assumption [RSW96]). We give a detailed explanation in Section 5.3.

Proof of work functions. A PoWF in the ROM is a tuple of algorithms $(\text{PoWF.Solve}, \text{PoWF.Verify})$. Given input x , PoWF.Solve outputs a solution y in with query complexity $q_{\text{PoWF.Solve}}$, and PoWF.Verify decides whether to accept y with query complexity $q_{\text{PoWF.Verify}}$. *Completeness* ensures that PoWF.Verify accepts the output of PoWF.Solve(x) with high probability. *Soundness* states that any algorithm with query complexity significantly below $q_{\text{PoWF.Solve}}$ can produce a valid y where $\text{PoWF.Verify}(x, y) = 1$ with only negligible probability. Similarly to VDFs, we additionally require PoWF to satisfy *computational uniqueness*, which means that no algorithm with query complexity $\text{poly}(q_{\text{PoWF.Solve}})$ can find $y' \neq \text{PoWF.Solve}(x)$ such that $\text{PoWF.Verify}(x, y') = 1$ with non-negligible probability.

We give a partial answer to the question raised before by showing that PoWFs that satisfies statistical uniqueness (no adversary can find different solutions for all but negligibly many inputs) do not exist in the ROM.

Theorem 2. *Suppose PoWF is a PoWF in the random oracle model that satisfies completeness and statistical uniqueness. Then, there exists an $O(q_{\text{PoWF.Verify}}^2)$ -query adversary Adv that breaks the soundness of PoWF.*

Theorem 2 leaves open the problem whether PoWFs with computational uniqueness can be constructed in the ROM. In fact, since the techniques we use to show Theorem 2 is closely related to certificate complexity and query complexity of search problems [KSS11; Smy02], we also provide an alternative formulation of this open problem in terms of search problems in Open Problem 1.

Table 1 below summarizes the current state of our understanding on the trade-off between uniqueness and sequentiality when constructing cryptographic primitives in the ROM.

	With Sequentiality	Without Sequentiality
Perfect Uniqueness	(perf. uniq. VDF) [MSW20], Theorem 1: ✗	(perf. uniq. PoWF) Theorem 2: ✗
Statistical Uniqueness	(stat. uniq. VDF) Theorem 1: ✗	(stat. uniq. PoWF) Theorem 2: ✗
Computational Uniqueness	(VDF) Theorem 1: ✗	(PoWF) Open
No Uniqueness	(PoSW) [DLM19]: ✓	(PoW) [GKL15]: ✓

Table 1: Trade-offs between uniqueness and sequentiality in the ROM. We use the red cross mark (✗) to indicate impossibility results and the blue check mark (✓) to indicate constructions.

1.2 Related works

VDF and related cryptographic primitives have been studied extensively in prior works. We summarize the works that are most relevant to our results.

Verifiable delay functions. [MSW20] shows that VDFs with perfect completeness (Eval outputs an accepting solution with probability 1) and adaptive perfect uniqueness cannot exist in the ROM. Our impossibility result on VDFs with imperfect completeness and non-adaptive computational uniqueness in the ROM is more general (see Theorem 3.4 and Theorem 3.5). In fact, we also show a stronger claim regarding VDFs with perfect uniqueness. We postpone a detailed comparison to Section 2.2. [DGMV20] presents an in-depth study of *tight VDFs*, a variant that requires the evaluation algorithm Eval to run in time almost the same as the sequentiality requirement, and proves a negative result in the ROM. Specifically, they show that there is no VDF construction that cannot be evaluated using T rounds of queries, but can be evaluated using $T + O(T^\delta)$ (for every constant $0 < \delta < 1$) rounds of queries. Note that this does not rule out VDFs that cannot be evaluated using T rounds of queries but can be evaluated using $T + \Omega(T)$ rounds of queries. [RSS20] shows that VDFs cannot be constructed in cyclic groups of known orders. In fact, their result works for generic-group delay functions, a generalization of VDFs. [EFKP20] constructs VDFs with perfect completeness and computational uniqueness in the ROM assuming the hardness of repeated squaring. We summarize the comparison between the above-mentioned works and our results in Table 2.

	Perfect Completeness	Imperfect Completeness
Perfect Uniqueness	[MSW20]: ✗ (ROM) Theorem 1: ✗ (ROM)	Theorem 1: ✗ (ROM)
Computational Uniqueness	[DGMV20]: ✗ (ROM + tight VDF) [RSS20]: ✗ (cyclic groups of known orders) [EFKP20]: ✓ (ROM + repeated squaring) Theorem 1: ✗ (ROM)	Theorem 1: ✗ (ROM)

Table 2: Comparison between prior works and our results. We use the red cross mark (✗) to indicate impossibility results and the blue check mark (✓) to indicate constructions. We specify the underlying assumption for each result in parenthesis.

Time-lock puzzles. Time-lock puzzles ([RSW96]) are similar to VDFs as they also have the uniqueness and sequentiality guarantee. In a time-lock puzzle, a generator outputs a puzzle x and a corresponding solution y efficiently. However, computing y from x still requires large sequential time. The main difference is that time-lock puzzles require the verifier to have knowledge of a *secret key* to achieve efficient verification, while

VDFs are publicly verifiable. [MMV11] rules out time-lock puzzles in the ROM.

Incrementally verifiable computations. Incrementally verifiable computation (IVC) [Val08] is a cryptographic primitive that enables efficient verification for multi-step computation. It is believed, though only partially proven [HN23; BCG24], that IVC does not exist in the ROM. [BBBF18] shows there is a black-box construction of VDFs from tight IVC (where IVC prover does not have too much overhead) for iterated sequential functions. Consequently, our results rule out tight IVC for iterated sequential functions in the ROM. However, since all hard sequential functions constructions use either the random oracle or cryptographic assumptions, our results imply that tight relativized IVC (tight IVC for which the target computation itself involves calls to the oracle) cannot be constructed in the ROM. In fact, [BCG24] proves a stronger claim: relativized IVC does not exist in the ROM, even when security holds against time-bounded (instead of just query-bounded) adversaries. We leave as an open question whether our techniques can be used to prove the impossibility of standard, non-relativized IVC in the ROM.

Proofs of work. A proof of work can be constructed in the ROM [GKL15]. Proof of sequential works (PoSWs) are proof of works with sequentiality guarantee [MMV13; CP18; AKKPW19; DLM19; AFGK22; AC23; Abu23]. It is known that PoSWs can be constructed in the ROM ([MMV13; CP18; DLM19]). Our results show that (i) VDFs, which can be viewed as proof of works with sequentiality and uniqueness, cannot be constructed in the ROM; (ii) statistically unique PoWFs, which is defined to be proof of works with statistical uniqueness, do not exist in the ROM.

2 Techniques

We overview the main ideas underlying our results. In Section 2.1, we discuss our reformulation of VDFs into search problems that enables us to apply techniques developed for decision tree algorithms, which enables us to use techniques in query complexity. In Section 2.2, we provide a simpler proof for the impossibility of VDFs with perfect uniqueness in the ROM as a warm-up. In Section 2.3, we explain how to generalize the approach in Section 2.2 to prove Theorem 2. In Section 2.4, we start with an alternative proof for the impossibility of VDFs with statistical uniqueness in the ROM and explain how to adapt it to show Theorem 1.

2.1 From VDFs to search problems

Review: VDF. A VDF in the ROM is a tuple of algorithms $\text{VDF} = (\text{Eval}, \text{Verify})$ that works as follows: for every security parameter $\lambda \in \mathbb{N}$, let the random oracle $\mathcal{O}(\lambda)$ be the uniform distribution over the set of all functions with output length λ ($\{f: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda\}$):

- The evaluation function Eval gets oracle access to a random oracle function f , receives an input $x \in \mathcal{X}$ and deterministically produces an output $y \in \mathcal{Y}$. (Note that Eval should also output a proof π , we omit it in this section for simplicity. Our formal proofs use the standard VDF definition as stated in Theorem 3.2.) Eval makes at most q_{Eval} queries to f .
- The verifier Verify gets oracle access to a random oracle function f , receives input $(x, y) \in \mathcal{X} \times \mathcal{Y}$ and deterministically decides whether to accept or reject. Verify makes at most q_{Verify} queries to f .

VDF is *complete* if the solution computed by Eval is accepted by Verify with high probability. For ease of discussion, we consider VDFs with perfect completeness in this section (imperfect completeness is handled carefully in Sections 4 and 5). VDF satisfies *sequentiality* if no r_{Adv} -round q_{Adv} -query ($r_{\text{Adv}} \ll q_{\text{Eval}}$ and $q_{\text{Adv}} = O(q_{\text{Eval}})$) algorithm can correctly compute Eval with non-negligible probability. Moreover, we say that VDF has *perfect uniqueness* if for every input x , Verify only accepts the output $y := \text{Eval}^f(x)$; VDF has *statistical uniqueness* if for every input x , Verify accepts an alternative output $y' \neq \text{Eval}^f(x)$ with negligible probability; VDF has *computational uniqueness* if for every input x and every poly(q_{Eval})-query adversary Adv, Verify accepts $\text{Adv}^f(x) \neq \text{Eval}^f(x)$ with negligible probability. Note that the above probabilities are with respect to the choice of the random oracle function f .

Review: search problems. A search problem $S \subseteq \mathcal{F} \times \mathcal{Y}$ is defined by a family of verifiers $\{\mathbb{V}_y: \mathcal{F} \rightarrow \{0, 1\}\}_{y \in \mathcal{Y}}$, where $(f, y) \in S$ if and only if $\mathbb{V}_y(f) = 1$. We say an algorithm $\mathbb{D}: \mathcal{F} \rightarrow \mathcal{Y}$ computes S if for every $f \in \mathcal{F}$, $(f, \mathbb{D}(f)) \in S$.

Reformulation of VDFs. Recall that every query algorithm can be viewed as a *decision tree*: the internal nodes of the tree represent the queries, the leaves represent the solutions, and the branching is based on the answers from the oracle to the queries.

In the ROM, the efficiency of the algorithms is measured by the number of queries they make to the random oracle. Thus, the execution of every sequential algorithm can be viewed as a decision tree. The same holds for parallel algorithms except that the internal nodes are now labeled by the set of queries instead of a single query.

Formally, fix a security parameter $\lambda \in \mathbb{N}$, for every $x \in \mathcal{X}$, we define a search problem $S_x \subseteq \mathcal{F} \times \mathcal{Y}$, where $\mathcal{F} := \{f: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda\}$ and $\mathcal{Y} := \mathcal{Y}$, by a family of verifiers $\{\mathbb{V}_y^{(x)}: \mathcal{F} \rightarrow \{0, 1\}\}_{y \in \mathcal{Y}}$ where for every $y \in \mathcal{Y}$,

$$\mathbb{V}_y^{(x)}(f) = \text{Verify}^f(x, y).$$

Moreover, we define $\mathbb{D}^{(x)} : F \rightarrow Y$, which computes S_x , such that for every $f \in F$,

$$\mathbb{D}^{(x)}(f) := \text{Eval}^f(x).$$

The convention in query complexity is that the domain of the relations is finite, whereas the domain $F = \{f : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda\}$ is infinite. We observe that it is sufficient to define $S_x \subseteq [2^\lambda]^n \times Y$ since there is some large constant n such that VDF depends on at most n positions of the random oracle. The total number of search problems we define is $|\mathcal{X}|$. For each search problem, we have at most $|\mathcal{Y}|$ verifiers of query complexity q_{Verify} , so each of them depends on at most $2^{\lambda q_{\text{Verify}}+1}$ positions in $\{0, 1\}^*$. Moreover, Eval has query complexity q_{Eval} , so it depends on at most $2^{\lambda q_{\text{Eval}}+1}$ points in the domain of f . Thus we can bound n by $n \leq |\mathcal{X}| (2^{\lambda q_{\text{Verify}}+1} |\mathcal{Y}| + 2^{\lambda q_{\text{Eval}}+1})$.

Hence, $\mathbb{V}_y^{(x)}$ has query complexity (with respect to the input string) $t := q_{\text{Verify}}$ and $\mathbb{D}^{(x)}$ has query complexity $T := q_{\text{Eval}}$. These search problems preserve many properties of the original VDF:

- Algorithms computing these search problems can be transformed into algorithms computing the original VDF with roughly the same complexity and success probability.
- VDFs with certain sequentiality and uniqueness properties correspond to search problems with similar properties.

2.2 Warm-up: VDFs with perfect uniqueness in the ROM

As a warm-up, we present a new proof for the impossibility of VDFs with perfect uniqueness in the ROM. Our proof is inspired by the classical algorithm witnessing that *decision tree complexity* is at most the square of *certificate complexity* for total boolean functions ([AB09]).

Since VDFs have perfect completeness and perfect uniqueness, we know that for every $x \in \mathcal{X}$ and $f \in [2^\lambda]^n$, there is a unique $y \in Y$ such that $(f, y) \in S_x$. Hence, according to the search problem reformulation outlined in Section 2.1, it suffices to prove the following lemma.

Lemma 1. *Let $S_x \subseteq [2^\lambda]^n \times Y$ be a search problem determined by a family of verifiers $\{\mathbb{V}_y^{(x)}\}_{y \in Y}$ of query complexity t such that for every $f \in [2^\lambda]^n$, $|\{y \in Y : (f, y) \in S_x\}| = 1$. Then there exists an $O(t)$ -round $O(t^2)$ -query adversary $\mathbb{A}^{(x)} : [2^\lambda]^n \rightarrow Y$ that computes S_x .*

For a fixed input $x \in \mathcal{X}$, let's consider the set of all accepting leaves $\{\ell_i\}_i$ of the verifiers $\{\mathbb{V}_y^{(x)}\}_{y \in Y}$. Note that each leaf ℓ_i is an element in $([2^\lambda] \cup \{\star\})^n$ such that for every $f \in [2^\lambda]^n$ that agrees with ℓ_i we have $\mathbb{V}_y^{(x)}(f) = 1$ for some $y \in Y$. For ease of notation, we define the domain $\text{dom}(\ell)$ for each $\ell \in ([2^\lambda] \cup \{\star\})^n$ as the set of positions that are determined:

$$\text{dom}(\ell) := \{i \in [n] : \ell[i] \neq \star\}.$$

For each $\ell \in ([2^\lambda] \cup \{\star\})^n$, we define its corresponding cube $\text{Cube}(\ell)$ as follows:

$$\text{Cube}(\ell) := \{f \in [2^\lambda]^n : \text{for all } q \in \text{dom}(\ell), f[q] = \ell[q]\}.$$

Since every $f \in [2^\lambda]^n$ has a unique solution y , $\text{Cube}(\ell_i)$'s are disjoint. Hence, for every $\ell_i \neq \ell_j$, there is some $q \in \text{dom}(\ell_i) \cap \text{dom}(\ell_j)$ such that $\ell_i[q] \neq \ell_j[q]$. In other words, if we pick an arbitrary ℓ_i and query the given random oracle function f at all positions in $\text{dom}(\ell_i)$, we “learn” at least one position for every leaf $\{\ell_i\}_i$. Since $\mathbb{V}_y^{(x)}$ makes at most q_{Verify} queries, each leaf contains at most q_{Verify} non- $\star$ positions. Thus, repeating the above process for q_{Verify} times suffices for an adversary to “learn” everything to determine the solution y . Hence, we can design an adversary that always outputs the correct solution y as follows:

1. Let L be the set of all accepting leaves $\{\ell_i\}_i$ of the verifiers $\{\mathbb{V}_y^{(x)}\}_{y \in Y}$.
2. Initialize $p^* := \star^n$.
3. For $i \in [q_{\text{Verify}}]$: Choose an arbitrary leaf ℓ in L . Query the given oracle function f at all positions in $\text{dom}(\ell)$. Update p^* to record the answers of f and remove from L every leaf inconsistent with p^* .
4. Output y where $\mathbb{V}_y^{(x)}(\ell) = 1$ for every $\ell \in \text{Cube}(p^*)$.

Observe that the adversary makes at most t^2 queries in t rounds. However, it is not computationally efficient since it needs to go over all accepting leaves, contrary to the adversaries we designed in the proof of Theorem 1 (see Section 5.3 for a detailed discussion).

Remark 1. [MSW20] proves VDFs with perfect completeness and perfect uniqueness do not exist in the ROM by constructing a sequentiality adversary that makes $(2q_{\text{Verify}} + 1) \cdot q_{\text{Eval}}$ queries in total over $2q_{\text{Verify}} + 1$ rounds. Qualitatively they show a similar result as Lemma 1. However, we construct a sequentiality adversary using only q_{Verify}^2 queries in q_{Verify} rounds and our construction works even when VDFs have imperfect completeness (see Section 4). In particular, this quantitative improvement on the query complexity of the adversary enables us to conclude that PoWFs with perfect uniqueness do not exist in the ROM.

2.3 Statistically unique PoWFs in the ROM

In this section we explain the idea behind Theorem 2. Observe that in Section 2.2 we have proved the following claim.

Claim 1 ([BI87], see also [AB09]). *For a given collection of certificates $\ell_1, \dots, \ell_N \in ([2^\lambda] \cup \{\star\})^n$ such that $\text{Cube}(\ell_i)$ are disjoint there exists a decision tree of depth $O((\max_{i \in [n]} |\text{dom}(\ell_i)|)^2)$ that finds the unique certificate ℓ_i that agrees with the given $f \in [2^\lambda]$.*

The only difference between the statement of Lemma 1 and Claim 1 is that instead of certificates we have a collection of verifiers. It is easy to see that they are equivalent: a certificate can be checked by a verifier and a verifier can be replaced with a collection of certificates corresponding to its accepting leaves.

Theorem 2 can be seen as an approximate analogue of Claim 1. In fact, as explained Remark 1, to rule out PoWFs with statistically uniqueness in the ROM, it is enough to construct a sequentiality adversary for statistically unique VDFs whose query complexity is polynomial in q_{Verify} .

The question of whether one can prove an approximate version of Claim 1 was first asked by Rudich in [Rud88], who derived from this (then) conjecture that one-way permutations do not exist in the random oracle model. Rudich's conjecture was resolved by Kahn, Saks, and Smyth [KSS11] using Berg-Kesten-Reimer (BKR) inequality, which was conjectured in [BK85] and proved in [Rei00].

[KSS11] taken together with [Rud88] implies that given a collection of certificates $\ell_1, \dots, \ell_N$, if a random point in $[2^\lambda]^n$ belongs to exactly one $\text{Cube}(\ell_i)$ with large enough constant probability, Claim 1 still applies, hence Theorem 2 holds. There is, however, an important technical caveat that prevents this simple logic to go through.

Ambiguity of a proof. As remarked in Section 2.1, the definition for VDFs we use in this section is simplified: in the actual definition following the standard definition of VDF, $\text{Eval}^f(x)$ outputs a pair (y, π) where π is a proof that can be used by Verify .¹ In the language of certificates it means that $\mathbb{V}_y^{(x)}$ corresponds to a collection of potentially non-disjoint certificates, as opposed to certificates corresponding to accepting leaves

¹We emphasize that although this simplification needs to be taken care of separately in this case, it does not affect the proof of Theorem 1 (see Sections 2.4 and 5).

of a single decision tree that are by definition disjoint. The uniqueness property of VDF then guarantees only that certificates *corresponding to different values* y have low total intersection, hence the result of [KSS11] does not apply.

Following up to [KSS11], Smyth [Smy02] proved that for two certificate collections $\ell_1, \dots, \ell_N$ and $\tau_1, \dots, \tau_L$ such that a random point belongs to exactly one of $\bigcup_{i \in [N]} \text{Cube}(\ell_i)$ and $\bigcup_{i \in [L]} \text{Cube}(\tau_i)$, there exists a decision tree of depth $O(t^2)$ finding a certificate agreeing with the given $f \in [2^\lambda]^n$, where t is the largest domain size among $\ell_1, \dots, \ell_N, \tau_1, \dots, \tau_L$. This would be exactly equivalent to Theorem 2 if $|Y| = 2$.

In order to prove Theorem 2, we generalize Smyth’s result to arbitrary many certificate (cube) families in Section 4. The proof follows the original one pretty closely. However, a black-box reduction to the case of two families seems elusive: a naïve attempt to partition the families in two groups, apply Smyth’s theorem and continue with a smaller group introduces an error multiplier logarithmic in the number of families, which is unacceptable in our case since the initial error is constant. Moreover, it is not even clear how to give any bound on $|Y|$ in terms of t .

2.4 VDFs with computational uniqueness in the ROM

We explain how to prove Theorem 1. In order to tackle VDFs with computational uniqueness, we start with a different approach to rule out VDFs with statistical uniqueness. In fact, our proof has two steps:

- Step 1: We construct an adversary that computes Eval with small sequential time if the given VDF admits statistical uniqueness;
- Step 2: We show that a modified adversary works well even when VDF only has computational uniqueness.

2.4.1 Adversary for VDFs with statistical uniqueness in the ROM

Similar to Section 2.1 we use our reformulation for a given VDF = (Eval, Verify). Note that now VDF only satisfies statistical uniqueness, we don’t expect our sequentiality adversary $\mathbb{A}^{(x)}$ to perfectly compute S_x anymore. Rather, we show that there exists some constant C such that for every $x \in \mathcal{X}$, there is a $O(t)$ -round $O(t \cdot T)$ -query adversary $\mathbb{A}^{(x)}$ that computes S_x with success probability at least $1 - C \cdot \epsilon$, where $\epsilon = \text{negl}(\lambda)$ is the uniqueness error of VDF.

Our proof is inspired by [MSW20, Algorithm 1], which they use to show that VDFs with perfect uniqueness cannot be constructed in the ROM. We first explain their idea and then present how we modify it to work in our setting. ([MSW20] presents their proof in terms of VDF, we rephrase it to fit into our decision tree framework.) For each input $x \in \mathcal{X}$, [MSW20] constructs an adversary $\mathbb{A}^{(x)}$ that proceeds in $2t + 1$ rounds to compute S_x . This adversary is described in Algorithm 1.

[MSW20] observes that in each iteration, if $\mathbb{A}^{(x)}(f)$ chooses a leaf that leads to some solution other than $y := \mathbb{D}^{(x)}(f)$, it queries at least one “new” position that has also been queried by $\mathbb{V}_y^{(x)}(f)$ in this iteration. Formally, let $\ell_{\mathbb{V},f}$ be the unique accepting leaf of $\mathbb{V}_y^{(x)}$ that contains f . Let p_i^* be the value of p^* at the beginning of iteration i . Since VDF satisfies perfect uniqueness, which means that for every chosen leaf ℓ_i such that $\mathbb{D}^{(x)}(\ell_i) \neq \mathbb{D}^{(x)}(f)$, the following holds:

$$\text{Cube}(p_i^* \cup \ell_i) \cap \text{Cube}(p_i^* \cup \ell_{\mathbb{V},f}) = \emptyset.$$

In other words, every time $\mathbb{A}^{(x)}(f)$ records a wrong solution, it makes progress in learning the verifier’s view of f . Since $\mathbb{V}_y^{(x)}$ has query complexity at most t , at most t of the recorded solutions do not equal to y , which implies that the majority of recorded solutions gives $y = \mathbb{D}^{(x)}(f)$.

Algorithm 1 Adversary $\mathbb{A}^{(x)}$ from [MSW20].

Input: $f \in [2^\lambda]^n$

Output: $y \in Y \cup \{\perp\}$

- 1: Let $L_1 := \{\ell_i\}_i$ be the set of leaves of $\mathbb{D}^{(x)}$.
 - 2: Initialize $p^* := \star^n$.
 - 3: Initialize $W := []$.
 - 4: **for** $i \in [2q_{\text{Verify}} + 1]$ **do**
 - 5: Choose an arbitrary leaf ℓ_i from L_i .
 - 6: Append $\mathbb{D}^{(x)}(\ell_i)$ to W .
 - 7: For every $q \in \text{dom}(\ell_i)$, query f at q and set $p^*[q] := f[q]$.
 - 8: Let $L_{i+1} \subseteq L_i$ be the set of all leaves in L_i that are consistent with p^* .
 - 9: **return** y if W contains some y that wins the majority vote; $\perp$ otherwise.
-

However, the above adversary cannot be directly applied in the statistical uniqueness setting: *the adversary might not make progress when it records a wrong solution.*

To be more specific, if the VDF does not have perfect uniqueness, when $\mathbb{A}^{(x)}(f)$ chooses a leaf ℓ_i that leads to some solution $y' \neq \mathbb{D}^{(x)}(f)$ in round i , it is possible that the following happens:

$$\text{Cube}(p_i^* \cup \ell_i) \cap \text{Cube}(p_i^* \cup \ell_{\mathbb{V},f}) \neq \emptyset.$$

Hence, we can neither record the correct solution nor learn the verifier's view of f in this case.

The above issue can be addressed by the following two modifications to the adversary $\mathbb{A}^{(x)}$:

- In each iteration, instead of choosing an arbitrary leaf ℓ_i from L_i , we need to carefully choose a leaf that “breaks less perfect uniqueness”. More specifically, we choose leaf ℓ_i such that $\text{Cube}(\ell_i) \cap \text{Cube}(p_i^*)$ contains fewer functions $f \in [2^\lambda]^n$ that have non-unique solutions in S_x than that in $\text{Cube}(p_i^*)$ (such leaf ℓ_i exists by an averaging argument).
- Our new adversary runs in $(2 + \delta)q_{\text{Verify}}$ rounds for some constant $\delta > 0$ instead of merely $2q_{\text{Verify}} + 1$ rounds.

As before, we know that there are at most t rounds i such that

$$\mathbb{D}^{(x)}(\ell_i) \neq \mathbb{D}^{(x)}(f) \text{ and } \text{Cube}(p_i^* \cup \ell_i) \cap \text{Cube}(p_i^* \cup \ell_{\mathbb{V},f}) = \emptyset.$$

Moreover, from statistical uniqueness, there are at most ϵ -fraction of $f \in [2^\lambda]^n$ such that there exists some $y' \in Y$ where $y' \neq \mathbb{D}^{(x)}$ and $\mathbb{V}_{y'}^{(x)} = 1$. By our specific choice of leaves in each round, in expectation, there are $(2 + \delta)q_{\text{Verify}} \cdot \epsilon$ rounds i such that

$$\mathbb{D}^{(x)}(\ell_i) \neq \mathbb{D}^{(x)}(f) \text{ and } \text{Cube}(p_i^* \cup \ell_i) \cap \text{Cube}(p_i^* \cup \ell_{\mathbb{V},f}) \neq \emptyset.$$

Hence, by Markov's inequality, $\mathbb{A}^{(x)}$ records the true solution in the majority of rounds with high probability.

2.4.2 Does computational uniqueness undermine the adversary?

We briefly discuss how the above adversary $\mathbb{A}^{(x)}$ would still succeed even when VDF satisfies only computational uniqueness. (We do need to modify $\mathbb{A}^{(x)}$ further in the formal proof, but the version outlined in Section 2.4.1 is good enough for an intuitive explanation.) The rigorous proof can be found in Section 5.

In order to better understand which part of the analysis outlined in Section 2.4.1 fails after relaxing the uniqueness guarantee, we first recall the difference in the definitions of statistical uniqueness and computational uniqueness:

- *Statistical uniqueness*: For a uniformly chosen $x \in \mathcal{X}$, there are at most ϵ -fraction of $f \in [2^\lambda]^n$ such that there exists some $y' \in Y$ where $y' \neq \mathbb{D}^{(x)}$ and $\mathbb{V}_{y'}^{(x)} = 1$.
- *Computational uniqueness*: For a uniformly chosen $x \in \mathcal{X}$ and every computationally-bounded adversary $\mathbb{B}^{(x)}$, there are at most ϵ -fraction of $f \in [2^\lambda]^n$ such that $\mathbb{B}$ can find some $y' \in Y$ where $y' \neq \mathbb{D}^{(x)}$ and $\mathbb{V}_{y'}^{(x)} = 1$.

According to the above definitions, for a VDF that satisfies computational uniqueness, it is possible that more than ϵ -fraction of $f \in [2^\lambda]^n$ admits multiple solutions. Hence, the previous analysis in Section 2.4.1 fails to work as we cannot directly bound the number of rounds such that $\mathbb{D}^{(x)}(\ell_i) \neq \mathbb{D}^{(x)}(f)$ and $\text{Cube}(p_i^* \cup \ell_i) \cap \text{Cube}(p_i^* \cup \ell_{\mathbb{V},f}) \neq \emptyset$ anymore.

Our key observation is that from such iterations we can extract non-canonical solutions for points in the intersection of $\text{Cube}(p_i^* \cup \ell_i)$ and $\text{Cube}(p_i^* \cup \ell_{\mathbb{V},f})$: by the choice of ℓ_i , the value of $\mathbb{D}^{(x)}$ for all these points is $\mathbb{D}^{(x)}(\ell_i)$; and by definition of $\ell_{\mathbb{V},f}$, the value $\mathbb{D}^{(x)}(f)$ is accepted by the verifier. In order to exploit this observation we devise a uniqueness adversary $\mathbb{B}^{(x)}$ coupled with the sequentiality adversary $\mathbb{A}^{(x)}$ in Section 2.4.1, in such a way that if there are too many rounds i where $\mathbb{D}^{(x)}(\ell_i) \neq \mathbb{D}^{(x)}(f)$ and $\text{Cube}(p_i^* \cup \ell_i) \cap \text{Cube}(p_i^* \cup \ell_{\mathbb{V},f}) \neq \emptyset$, $\mathbb{B}^{(x)}$ breaks computational uniqueness. Since $\mathbb{B}^{(x)}$ needs to work with non-negligible probability for a uniformly random function $f \in [2^\lambda]^n$, we have to modify the sequentiality adversary $\mathbb{A}^{(x)}$ such that the non-uniqueness witnesses are distributed uniformly.

We carefully explain how one can modify the construction of $\mathbb{A}^{(x)}$ and construct an effective uniqueness adversary $\mathbb{B}^{(x)}$ to rule out VDFs with computational uniqueness in Section 5.

Open Problem 1. *We do not know whether it is possible to construct a sequentiality adversary for computationally unique VDFs with query complexity polynomially related to q_{Verify} . In fact, the existence of such an adversary would imply that PoWFs cannot be constructed in the ROM. We present a query-complexity formulation of this problem below.*

Let $S \subseteq F \times Y$ be a search problem defined by a verifier $\mathbb{V}: F \times Y \rightarrow \{0, 1\}$ with query complexity t . Let $\mathbb{D}: F \rightarrow Y$ be a T -query algorithm (with arbitrarily many rounds) that computes S correctly with high probability. Assume that for uniformly sampled $f \in F$, it is computationally hard to find $y' \neq \mathbb{D}(f)$ where $\mathbb{V}(f, y') = 1$. Is it possible to construct a $\text{poly}(t)$ -query adversary that computes S with non-negligible probability?

3 Preliminaries

Definition 3.1 (The random oracle model (ROM)). *For every $\lambda \in \mathbb{N}$, the random oracle $\mathcal{O}(\lambda)$ is the uniform distribution over the set of all functions $f: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$.*

3.1 Verifiable delay functions in the random oracle model

Definition 3.2 (Verifiable delay function (VDF) [BBBF18] in the random oracle model). A **verifiable delay function VDF in the ROM** is a tuple of oracle-aided algorithms $\text{VDF} = (\text{Setup}, \text{Eval}, \text{Verify})$ such that for every $\lambda \in \mathbb{N}$ and $f \in \mathcal{O}(\lambda)$, the following hold:

- $\text{Setup}^f(1^\lambda, q_{\text{Eval}}) \rightarrow \text{pp}$: On input the security parameter λ and the query bound q_{Eval} , the **deterministic** setup algorithm Setup outputs the public parameters pp , where pp determines a (uniformly) samplable input space $\mathcal{X}$ and an output space $\mathcal{Y}$.
- $\text{Eval}^f(\text{pp}, x) \rightarrow (y, \pi)$: On input the public parameter pp and an element $x \in \mathcal{X}$, the evaluation algorithm Eval outputs y and a proof π , where y is generated **deterministically** while π can be generated in a randomized way. (Specifically, Eval outputs y before it accesses its internal randomness.) We sometimes ignore the output proof π and write $\text{Eval}^f(\text{pp}, x) \rightarrow y$ for simplicity.
- $\text{Verify}^f(\text{pp}, x, y, \pi) \rightarrow \{0, 1\}$: On input the public parameter pp , and element $x \in \mathcal{X}$, a value $y \in \mathcal{Y}$, and a proof π , the **deterministic** verification algorithm Verify outputs a bit indicating whether it accepts or rejects.

We require that Setup , Eval and Verify make at most q_{Setup} , q_{Eval} and q_{Verify} queries, respectively, to the random oracle, where $q_{\text{Setup}} = q_{\text{Setup}}(\lambda, q_{\text{Eval}})$ and $q_{\text{Verify}} = q_{\text{Verify}}(\lambda, q_{\text{Eval}})$. In practice, we want to have VDFs where $q_{\text{Setup}} \ll q_{\text{Eval}}$ and $q_{\text{Verify}} \ll q_{\text{Eval}}$.

Definition 3.3 (Completeness of VDF). $\text{VDF} = (\text{Setup}, \text{Eval}, \text{Verify})$ has completeness error α if for every $\lambda \in \mathbb{N}$ and $q_{\text{Eval}} \in \mathbb{N}$,

$$\Pr \left[\text{Verify}^f(\text{pp}, x, y, \pi) = 1 \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{Setup}^f(1^\lambda, q_{\text{Eval}}) \\ x \leftarrow \mathcal{X} \\ (y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x) \end{array} \right] \geq 1 - \alpha(\lambda).$$

When $\alpha = 0$, we say the VDF has **perfect completeness**.

Definition 3.4 (Non-adaptive $(q_{\text{Adv}}, \epsilon)$ -uniqueness of VDF). For every q_{Adv} and ϵ , $\text{VDF} = (\text{Setup}, \text{Eval}, \text{Verify})$ satisfies $(q_{\text{Adv}}, \epsilon)$ -uniqueness if for every $\lambda \in \mathbb{N}$, $q_{\text{Eval}} \in \mathbb{N}$, and q_{Adv} -query adversary Adv ,

$$\Pr \left[\begin{array}{l} y \neq \text{Eval}^f(\text{pp}, x) \\ \wedge \text{Verify}^f(\text{pp}, x, y, \pi) = 1 \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{Setup}^f(1^\lambda, q_{\text{Eval}}) \\ x \leftarrow \mathcal{X} \\ (y, \pi) \leftarrow \text{Adv}^f(\text{pp}, x) \end{array} \right] \leq \epsilon(\lambda).$$

We say that VDF satisfies **perfect uniqueness** if q_{Adv} is unbounded and $\epsilon(\lambda) = 0$. We say that VDF satisfies **statistical uniqueness** if q_{Adv} is unbounded and $\epsilon(\lambda) = \text{negl}(\lambda)$. We say that VDF satisfies **computational uniqueness** if $q_{\text{Adv}} = \text{poly}(\lambda, q_{\text{Eval}})$ and $\epsilon(\lambda) = \text{negl}(\lambda)$.

Remark 3.5. Note that in previous works (e.g. [BBBF18; MSW20; DGMV20]), uniqueness is defined adaptively. In other words, instead of sampling an input x uniformly at random and giving to the adversary Adv as input, they allow Adv to choose the input themselves. The adaptive uniqueness is a stronger security notion than our non-adaptive uniqueness. However, since our focus in this paper is on impossibility results, we work with non-adaptive uniqueness, which implies stronger impossibility results compared to their adaptive analogues. We sometimes write “uniqueness” instead of “non-adaptive uniqueness” for simplicity; however, we always write “adaptive uniqueness” explicitly.

Definition 3.6 ($(r_{\text{Adv}}, q_{\text{Adv}}, \gamma)$ -sequentiality of VDF). *For every $r_{\text{Adv}}, q_{\text{Adv}}$, and γ , $\text{VDF} = (\text{Setup}, \text{Eval}, \text{Verify})$ is $(r_{\text{Adv}}, q_{\text{Adv}}, \gamma)$ -sequential if for every $\lambda \in \mathbb{N}$, $r_{\text{Adv}} \in \mathbb{N}$, $q_{\text{Eval}} \in \mathbb{N}$, and r_{Adv} -round q_{Adv} -query adversary Adv,*

$$\Pr \left[y = \text{Eval}^f(\text{pp}, x) \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{Setup}^f(1^\lambda, q_{\text{Eval}}) \\ x \leftarrow \mathcal{X} \\ y \leftarrow \text{Adv}^f(\text{pp}, x) \end{array} \right] \leq \gamma(\lambda).$$

Remark 3.7. Note that we allow the adversary in the sequentiality definition to be parallel algorithms: it can ask multiple queries in the same round, as long as the total number of queries across rounds is upper bounded by q_{Adv} . Moreover, canonical VDF definitions (e.g. [BBBF18]) require γ to be negligible in λ , here we consider the more general definition that considers various γ .

3.2 Proof of work functions in the random oracle model

Definition 3.8 (Proof of work function (PoWF) in the random oracle model). **A proof of work function PoWF in the ROM is a tuple of oracle-aided algorithms $\text{PoWF} = (\text{PoWF.Setup}, \text{PoWF.Solve}, \text{PoWF.Verify})$ such that for every $\lambda \in \mathbb{N}$ and $f \in \mathcal{O}(\lambda)$, the following hold:**

- $\text{PoWF.Setup}^f(1^\lambda, q_{\text{PoWF.Solve}}) \rightarrow \text{pp}$: *On input the security parameter λ and the query bound $q_{\text{PoWF.Solve}}$, the **deterministic** setup algorithm PoWF.Setup outputs the public parameters pp , where pp determines a (uniformly) samplable input space $\mathcal{X}$ and an output space $\mathcal{Y}$.*
- $\text{PoWF.Solve}^f(\text{pp}, x) \rightarrow (y, \pi)$: *On input the public parameter pp and an element $x \in \mathcal{X}$, the solve algorithm PoWF.Solve outputs a solution y and a proof π , where y is generated **deterministically** while π can be generated in a randomized way. We sometimes ignore the output proof π and write $\text{PoWF.Solve}^f(\text{pp}, x) \rightarrow y$ for simplicity.*
- $\text{PoWF.Verify}^f(\text{pp}, x, y, \pi) \rightarrow \{0, 1\}$: *On input the public parameter pp , element $x \in \mathcal{X}$, a value $y \in \mathcal{Y}$ and a proof π , the **deterministic** verification algorithm PoWF.Verify outputs a bit indicating whether it accepts or rejects.*

We require that PoWF.Setup , PoWF.Solve and PoWF.Verify make at most $q_{\text{PoWF.Setup}}$, $q_{\text{PoWF.Solve}}$ and $q_{\text{PoWF.Verify}}$ queries, respectively, to the random oracle, where $q_{\text{PoWF.Setup}} = q_{\text{PoWF.Setup}}(\lambda, q_{\text{PoWF.Solve}})$ and $q_{\text{PoWF.Verify}} = q_{\text{PoWF.Verify}}(\lambda, q_{\text{PoWF.Solve}})$. In practice, we want to have PoWFs where $q_{\text{PoWF.Setup}} \ll q_{\text{PoWF.Solve}}$ and $q_{\text{PoWF.Verify}} \ll q_{\text{PoWF.Solve}}$.

Definition 3.9 (Completeness of PoWF). $\text{PoWF} = (\text{PoWF.Setup}, \text{PoWF.Solve}, \text{PoWF.Verify})$ has completeness error α if for every $\lambda \in \mathbb{N}$ and $q_{\text{PoWF.Solve}} \in \mathbb{N}$,

$$\Pr \left[\text{PoWF.Verify}^f(\text{pp}, x, y, \pi) = 1 \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{PoWF.Setup}^f(1^\lambda, q_{\text{PoWF.Solve}}) \\ x \leftarrow \mathcal{X} \\ (y, \pi) \leftarrow \text{PoWF.Solve}^f(\text{pp}, x) \end{array} \right] \geq 1 - \alpha(\lambda).$$

When $\alpha = 0$, we say the PoWF has **perfect completeness**.

Definition 3.10 ($(q_{\text{PoWF.Adv}}, \kappa)$ -soundness of PoWF). $\text{PoWF} = (\text{PoWF.Setup}, \text{PoWF.Solve}, \text{PoWF.Verify})$ has $(q_{\text{PoWF.Adv}}, \kappa)$ -soundness κ if for every $\lambda \in \mathbb{N}$, $q_{\text{PoWF.Solve}} \in \mathbb{N}$, and $q_{\text{PoWF.Adv}}$ -query adversary PoWF.Adv ,

$$\Pr \left[y = y' \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{PoWF.Setup}^f(1^\lambda, q_{\text{PoWF.Solve}}) \\ x \leftarrow \mathcal{X} \\ (y, \pi) \leftarrow \text{PoWF.Solve}^f(\text{pp}, x) \\ y' \leftarrow \text{PoWF.Adv}^f(\text{pp}, x) \end{array} \right] \leq \kappa(\lambda).$$

Definition 3.11 (Non-adaptive $(q_{\text{PoWF.Adv}}, \epsilon)$ -uniqueness of PoWF). For every $q_{\text{PoWF.Adv}} \in \mathbb{N}$ and ϵ , $\text{PoWF} = (\text{PoWF.Setup}, \text{PoWF.Solve}, \text{PoWF.Verify})$ satisfies $(q_{\text{PoWF.Adv}}, \epsilon)$ -uniqueness if for every $\lambda \in \mathbb{N}$, $q_{\text{PoWF.Solve}} \in \mathbb{N}$, and $q_{\text{PoWF.Adv}}$ -query adversary PoWF.Adv ,

$$\Pr \left[\begin{array}{l} y \neq y' \\ \wedge \text{PoWF.Verify}^f(\text{pp}, x, y', \pi') = 1 \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{PoWF.Setup}^f(1^\lambda, q_{\text{PoWF.Solve}}) \\ x \leftarrow \mathcal{X} \\ (y, \pi) \leftarrow \text{PoWF.Solve}^f(\text{pp}, x) \\ (y', \pi') \leftarrow \text{PoWF.Adv}^f(\text{pp}, x) \end{array} \right] \leq \epsilon(\lambda).$$

We say that PoWF satisfies **perfect uniqueness** if $q_{\text{PoWF.Adv}}$ is unbounded and $\epsilon(\lambda) = 0$. We say that PoWF satisfies **statistical uniqueness** if $q_{\text{PoWF.Adv}}$ is unbounded and $\epsilon(\lambda) = \text{negl}(\lambda)$. We say that PoWF satisfies **computational uniqueness** if $q_{\text{PoWF.Adv}} = \text{poly}(\lambda, q_{\text{PoWF.Solve}})$ and $\epsilon(\lambda) = \text{negl}(\lambda)$.

3.3 Search problems

Definition 3.12. A search problem is defined by a relation $S \subseteq \mathcal{F} \times \mathcal{Y}$. We say S is **determined** by a family of nondeterministic verifiers $\{\mathbb{V}_{y,\pi}\}_{y \in \mathcal{Y}, \pi \in \Pi}$ if for every $f \in \mathcal{F}$, $y \in \mathcal{Y}$, $(f, y) \in S$ if and only if there exists some $\pi \in \Pi$ such that $\mathbb{V}_{y,\pi}(f) = 1$. We say a search problem is **total** if, for every $f \in \mathcal{F}$, there is at least one solution y s.t. $(f, y) \in S$.

We focus on search problems with product input space $\mathcal{F} = [M]^m$ for $M, m \in \mathbb{N}$. Given $f \in [M]^m$, $I \subseteq [m]$, $p \in [M]^I$, we define $f_{I \rightarrow p} \in [M]^m$ as follows:

$$f_{I \rightarrow p}[i] := \begin{cases} p[i] & i \in I \\ f[i] & i \notin I \end{cases}.$$

Definition 3.13 (Subcube). Fix $M, m \in \mathbb{N}$. Let $\mathcal{F} = [M]^m$. We say $\mathcal{F}' \subseteq \mathcal{F}$ is a **(sub)cube** if $\mathcal{F}' = \mathcal{F}'_1 \times \dots \times \mathcal{F}'_m$ for some $\mathcal{F}'_1, \dots, \mathcal{F}'_m \subseteq [M]$, where $|\mathcal{F}'_i| \in \{1, M\}$ for each $i \in [m]$.

Every query algorithm can be viewed as a **decision tree**: the internal nodes of the tree represent the queries, the leaves represent the solutions, and the branching is based on the answers from the oracle to the queries.

A **partial assignment** $p \in ([M] \cup \{\star\})^m$ is a length- m string, where each entry is either fixed to be some value in $[M]$, or “undetermined” (denoted by $\star$). The **domain of** p is defined as $\text{dom}(p) := \{i : p_i \neq \star\}$.

For ease of notation, we also identify each node p in a decision tree with a partial assignment $p \in ([M] \cup \{\star\})^m$ that records the query outcomes leading to the node p , if a position i is not queried, we set $p_i := \star$.

We say an input $f \in [M]^m$ is **consistent with a partial assignment** p if they agree on the domain of p , i.e. $f[i] = p[i]$ for all $i \in \text{dom}(p)$. We denote by $\text{Cube}(p) := \{f \in [M]^m : \forall i \in \text{dom}(p), f[i] = p[i]\}$ the set of all inputs consistent with p .

We say that **partial assignments** p and q are **consistent with each other** if they agree on every position in the intersection of their domains, i.e. for every $i \in \text{dom}(p) \cap \text{dom}(q)$ we have $p[i] = q[i]$. Equivalently, $\text{Cube}(p) \cap \text{Cube}(q) \neq \emptyset$. We use $p \cup q$ to denote the partial assignment with domain $\text{dom}(p) \cup \text{dom}(q)$ that is consistent with both p and q . Note that $\text{Cube}(p) \cap \text{Cube}(q) = \text{Cube}(p \cup q)$.

Given a distribution μ over some space F , for each $F' \subseteq F$, we define $\mu(F') := \sum_{x \in F'} \mu(x)$ as the probability of a random element sampled from μ is in F' . We use $\mathcal{U}_F$ to denote the uniform distribution over F .

For any two partial assignments $p, q \in ([M] \cup \{\star\})^m$, we say p and q are **independent**, denoted $p \approx^d q$, if $\text{dom}(p) \cap \text{dom}(q) = \emptyset$. Otherwise, we say that p and q are **dependent**, denoted $p \sim^d q$.

Theorem 3.14 (BKR inequality). [BK85; Rei00] Let P, Q be two collections of partial assignments over $[M]^m$. Then for every product distribution μ over $[M]^m$,

$$\mu^{\otimes 2} \left(\bigcup_{\substack{p \in P, q \in Q \\ p \sim^d q}} \text{Cube}(p) \times \text{Cube}(q) \right) \leq \mu \left(\bigcup_{\substack{p \in P \\ q \in Q}} \text{Cube}(p) \cap \text{Cube}(q) \right).$$

3.4 VDFs to search problems

Consider VDF = (Setup, Eval, Verify) with completeness error α . We present the formal reformulation of VDF in terms of search problems as described in Section 2.1. Note that this reformulation directly applies to PoWF.

Fix $\lambda \in \mathbb{N}$ and a large enough constant n that depends on q_{Setup} , q_{Eval} and q_{Verify} . The search problems are defined below:

For every leaf $\ell \in ([2^\lambda] \cup \{\star\})^n$ of the decision tree representation of Setup:

- (a) Let pp denote the label of ℓ . Deduce $\mathcal{X}$ and $\mathcal{Y}$ from pp .
- (b) For every $x \in \mathcal{X}$, define the search problem $S_{\ell, x} \subseteq \text{Cube}(\ell) \times \mathcal{Y}$ where $\mathcal{Y} := \mathcal{Y}$ as follows:
 - i. $S_{\ell, x}$ is determined by verifiers $\mathbb{V}_{y, \pi} : \text{Cube}(\ell) \rightarrow \{0, 1\}$ of query complexity q_{Verify} which satisfy that $\mathbb{V}_{y, \pi}(f) = \text{Verify}^f(\text{pp}, x, y, \pi)$.
 - ii. There is an algorithm $\mathbb{D} : \text{Cube}(\ell) \rightarrow \mathcal{Y} \times \Pi$ of query complexity q_{Eval} which satisfies that $\mathbb{D}(f) = \text{Eval}^f(\text{pp}, x)$ and $\Pr_{f \leftarrow [M]^m} [\mathbb{V}_{\mathbb{D}(f)}(f) = 1] \geq 1 - \alpha_{\ell, x}$ for some $\alpha_{\ell, x} \in [0, 1]$.

Moreover, for every f , let $\ell_{S,f}$ denote the leaf of the decision tree representation of Setup such that $f \in \text{Cube}(\ell_{S,f})$. It follows from Theorem 3.3 that

$$\mathbb{E} \left[\alpha_{\ell_{S,f},x} \left| \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{Setup}^f(1^\lambda, \mathbf{q}_{\text{Eval}}) \\ x \leftarrow \mathcal{X} \end{array} \right. \right] \leq \alpha.$$

4 PoWFs with statistical uniqueness

Theorem 4.1 (Formal version of Theorem 2). *Suppose $\text{PoWF} = (\text{PoWF.Setup}, \text{PoWF.Solve}, \text{PoWF.Verify})$ is a PoWF in the ROM with completeness error α that satisfies statistical uniqueness with error ϵ . Fix $\lambda \in \mathbb{N}$. Let $q_{\text{PoWF.Setup}}$ and $q_{\text{PoWF.Verify}}$ denote the query complexity of PoWF.Setup and PoWF.Verify , respectively. Then for every non-zero constant δ such that $\alpha + \epsilon \leq \delta \leq 10^{-3}$, PoWF does not satisfy $(q_{\text{PoWF.Setup}} + q_{\text{PoWF.Verify}}^2/\delta, 1 - 6\sqrt{\delta})$ -soundness.*

According to Section 3.4, it suffices to prove the theorem below.

Theorem 4.2. *Let $S \subseteq [M]^m \times Y$ be a search problem determined by a family of nondeterministic verifiers $\{\mathbb{V}_{y,\pi}\}_{y \in Y, \pi \in \Pi}$ of query complexity t . Let $\delta \geq 0$ be a parameter such that*

$$\Pr_{f \leftarrow [M]^m} [|\{y \in Y \mid (f, y) \in S\}| \neq 1] \leq \delta$$

Then for $\delta \leq 10^{-3}$, there exists a t/δ -round and t^2/δ -query adversary $\mathbb{A}: [M]^m \rightarrow Y$ such that

$$\Pr_{f \leftarrow [M]^m} [(f, \mathbb{A}(f)) \in S] \geq 1 - 6\sqrt{\delta}.$$

Proof of Theorem 4.1 by Theorem 4.2. Observe that δ in Theorem 4.2 bounds from above the sum of the uniqueness and completeness errors, i.e., $\alpha + \epsilon \leq \delta$. Let $\mathbb{A}$ be the adversary in Theorem 4.2. We construct a VDF sequentiality adversary PoWF.Adv that gets oracle access to f , runs PoWF.Setup to locate the leaf ℓ_f , samples $x \leftarrow \mathcal{X}$, and executes the adversary $\mathbb{A}$ corresponding to the search problem $S_{\ell_f, x}$. It follows that PoWF.Adv makes $(q_{\text{PoWF.Setup}} + q_{\text{PoWF.Verify}}^2/\delta)$ queries across $(q_{\text{PoWF.Setup}} + q_{\text{PoWF.Verify}}/\delta)$ round and correctly computes PoWF.Solve with probability at least $1 - 6\sqrt{\delta}$. $\square$

4.1 Proof of Theorem 4.2

Given a set $F' \subseteq [M]^m$, we define $\text{Uniq}(F') := \{f \in F' : |\{y \in Y \mid (f, y) \in S\}| = 1\}$ as the set of inputs in F' which have a unique solution w.r.t. S . We describe the adversary $\mathbb{A}$ in Algorithm 2. The general structure of the algorithm follows the one described in Section 2.2. For several rounds we pick a leaf of a verifier that is consistent with the current assignment and query the input in the positions corresponding to that leaf. The crucial part of the algorithm is to choose that leaf correctly.

Algorithm 2 Sequentiality-breaking adversary $\mathbb{A}$ for statistical uniqueness.

Input: $f \in [M]^m$

Output: $y \in Y \cup \{\perp\}$

- 1: Initialize $p^* := \star^n$, $y_0 := \perp$.
 - 2: **for** $i \in [t/\delta + 1]$ **do**
 - 3: **for** $y \in Y$ **do**
 - 4: **if** $\mathcal{U}_{\text{Cube}(p^*)}(\{f' \in \text{Cube}(p^*) : (f', y) \in S\}) \geq 1 - 3\sqrt{\delta}$ **then return** y .
 - 5: **if** $\mathcal{U}_{\text{Cube}(p^*)}(\text{Uniq}(\text{Cube}(p^*))) < 1 - \sqrt{\delta}/2$ **then return** $\perp$.
 - 6: **if** $i = t/\delta + 1$ **then return** $\perp$.
 - 7: Set $\ell_i := \text{FINDBESTLEAF}(p^*)$, where FINDBESTLEAF is constructed in Algorithm 3.
 - 8: For every $q \in \text{dom}(\ell_i)$, query f at q and set $p^*[q] := f[q]$.
-

There are three cases where $\mathbb{A}$ outputs an invalid solution:

1. Algorithm 2 halts at Line 4 but returns a solution y such that $(f, y) \notin S$.
2. Algorithm 2 halts at Line 5 and returns $\perp$.
3. Algorithm 2 halts at Line 6 and returns $\perp$.

We denote the above three events by E_1, E_2, E_3 respectively. We prove that each of them happens with low probability (over random $\mathbf{f} \leftarrow [M]^m$).

First we bound the probability of E_1 and E_2 . They do not depend on the definition of `FINDBESTLEAF`. Let $L_{\mathbb{A}}$ denote the set of leaves of the decision tree representation of $\mathbb{A}$. Observe that for each fixed leaf $\ell \in L_{\mathbb{A}}$, $\mathbb{A}$ behaves identically for all $f \in \text{Cube}(\ell)$.

Claim 4.3. $\Pr_{\mathbf{f} \leftarrow [M]^m}[E_1] \leq 3\sqrt{\delta}$.

Proof. For each leaf $\ell \in L_{\mathbb{A}}$ such that $\mathbb{A}$ terminates on Line 4, let $y(\ell)$ denote the solution that $\mathbb{A}$ outputs. We have $\Pr_{\mathbf{f} \sim \text{Cube}(\ell)}[E_1] = \Pr_{\mathbf{f} \sim \text{Cube}(\ell)}[(\mathbf{f}, y(\ell)) \notin S] \leq 3\sqrt{\delta}$, where the inequality follows from the if condition on Line 4. By averaging over all the leaves in $L_{\mathbb{A}}$, we obtain the desired claim. $\square$

Claim 4.4. $\Pr_{\mathbf{f} \leftarrow [M]^m}[E_2] \leq 2\sqrt{\delta}$.

Proof. Let $L_{\mathbb{A}}^{(2)} \subseteq L_{\mathbb{A}}$ denote the set of leaves that lead to E_2 . Observe that

$$\begin{aligned} \delta &\geq 1 - \mathcal{U}_{[M]^m}(\text{Uniq}([M]^m)) \\ &\geq \sum_{\ell \in L_{\mathbb{A}}^{(2)}} \mathcal{U}_{[M]^m}(\text{Cube}(\ell) \setminus \text{Uniq}(\text{Cube}(\ell))) \\ &\geq \frac{\sqrt{\delta}}{2} \cdot \sum_{\ell \in L_{\mathbb{A}}^{(2)}} \mathcal{U}_{[M]^m}(\text{Cube}(\ell)), \end{aligned}$$

where the last inequality follows from the if condition on Line 5. As a consequence,

$$\Pr_{\mathbf{f} \leftarrow [M]^m}[E_2] = \sum_{\ell \in L_{\mathbb{A}}^{(2)}} \mathcal{U}_{[M]^m}(\text{Cube}(\ell)) \leq 2\sqrt{\delta}. \quad \square$$

Claim 4.5. $\Pr_{\mathbf{f} \leftarrow [M]^m}[E_3] \leq \sqrt{\delta}$.

Given Theorem 4.5 we can conclude that $\mathbb{A}$ makes t^2/δ queries in t/δ rounds and succeeds with probability at least $1 - 6\sqrt{\delta}$.

4.2 Finding the best leaf: proof of Theorem 4.5

Event E_3 can be described as “ $\mathbb{A}$ fails to terminate in t/δ iterations”. Observe that $\mathbb{A}$ must halt in Algorithm 2 if $\text{Cube}(p^*)$ is contained within $\text{Cube}(\ell)$ where ℓ is an assignment corresponding to any leaf of a verifier $\mathbb{V}_{y,\pi}$. We introduce a potential function to measure how far we are from this situation. Let

$$L := \{\ell \in ([M] \cup \{\star\})^m : \ell \text{ is a leaf of } \mathbb{V}_{y,\pi} \text{ for some } y \in Y, \pi \in \Pi\}.$$

Let $L_{p^*} := \{\ell \in L : \ell \text{ agrees with } p^*\}$. For $i \in [t/\delta]$ the potential function w_i is defined as

$$w_i(f) := \begin{cases} \min_{\ell \in L_{p^*} : f \in \text{Cube}(\ell)} |\text{dom}(\ell) \setminus \text{dom}(p^*)| & f \in \text{Uniq}(\text{Cube}(p^*)) \\ 0 & f \notin \text{Uniq}(\text{Cube}(p^*)) \end{cases},$$

where p^* is the value of this variable in $\mathbb{A}(f)$ at the i -th iteration. For all f such that $\mathbb{A}(f)$ halts before invoking **FINDBESTLEAF** in the i -th iteration, we define $w_i(f) := 0$. The routine for **FINDBESTLEAF** we describe in Algorithm 3 can be informally summarized as: find ℓ such that the potential function decreases for as many values of $f \in \text{Cube}(p^*)$ as possible.

Algorithm 3 Subroutine **FINDBESTLEAF** in Algorithm 2

Input: $p^* \in ([M] \cup \{\star\})^m$

Output: $\ell \in L_{p^*}$

- 1: **for** $f' \in \bigcup_{\ell \in L_{p^*}} \text{Cube}(\ell \cup p^*)$ **do**
 - 2: Set $\min \ell(f') := \arg \min_{\ell \in L_{p^*}: f' \in \text{Cube}(\ell \cup p^*)} |\text{dom}(\ell) \setminus \text{dom}(p^*)|$.
 - 3: **for** $\ell \in L$ **do**
 - 4: Set $C_\ell := \{f \in \text{Cube}(\ell \cup p^*) : \min \ell(f) = \ell\}$.
 - 5: **return** $\arg \max_{\ell \in L_{p^*}} |\bigcup_{\ell' \in L_{p^*}: \ell' \sim^d \ell} C_{\ell'}|$.
-

For each $i \in [t/\delta]$, let P_i denote the set of all possible query outcomes p^* that are passed to **FINDBESTLEAF** in the i -th iteration. Fix any $p^* \in P_i$. For ease of notation, we abbreviate $\mathcal{U}_{\text{Cube}(p^*)}$ as $\mathcal{U}$ in the rest of the proof.

For each $\ell \in L_{p^*}$, define

$$F_{p^*}(\ell) := \bigcup_{\substack{\ell' \in L_{p^*} \\ \ell' \sim^d \ell}} C_{\ell'},$$

where $C_{\ell'}$ is defined at Line 4 in Algorithm 3. We prove that

Claim 4.6. *There exists some $\hat{\ell} \in L_{p^*}$ such that $\mathcal{U}(F_{p^*}(\hat{\ell})) \geq (3/2)\sqrt{\delta}$.*

We defer the proof of Theorem 4.6 to Section 4.3. By Theorem 4.6 we have $\mathcal{U}(F_{p^*}(\ell_i)) \geq (3/2)\sqrt{\delta}$ since $\ell = \ell_i$ maximizes $\mathcal{U}(F_{p^*}(\ell))$ by our choice of ℓ_i (Line 5).

Let $G(p^*) := F_{p^*}(\ell_i) \cap \text{Uniq}(\text{Cube}(p^*))$. Observe that for all $f \in G(p^*)$, we have $w_{i+1}(f) \leq w_i(f) - 1$ since $\text{dom}(\ell_i) \setminus \text{dom}(p^*)$ has intersection with $\text{dom}(\min \ell(f)) \setminus \text{dom}(p^*)$. Moreover,

$$\mathcal{U}(G(p^*)) \geq \mathcal{U}(F_{p^*}(\ell_i)) - (1 - \mathcal{U}(\text{Uniq}(\text{Cube}(p^*)))) \geq (3/2)\sqrt{\delta} - \sqrt{\delta}/2 \geq \sqrt{\delta}.$$

Define $H_i := \mathbb{E}_{f \leftarrow [M]^m} [w_i(f)]$. Let $\eta_i := \sum_{p^* \in P_i} \mathcal{U}_{[M]^m}(\text{Cube}(p^*))$ denote the fraction of inputs that survive after the i -th iteration. Then

$$\begin{aligned} H_{i+1} - H_i &\geq \sum_{p^* \in P_i} \mathcal{U}_{[M]^m}(\text{Cube}(p^*)) \cdot \mathcal{U}_{\text{Cube}(p^*)}(G(p^*)) \\ &\geq \sqrt{\delta} \sum_{p^* \in P_i} \mathcal{U}_{[M]^m}(\text{Cube}(p)) \\ &= \sqrt{\delta} \eta_i. \end{aligned}$$

Observe that $\eta_1 \geq \dots \geq \eta_{t/\delta} \geq \Pr_{f \leftarrow [M]^m} [E_3]$. Moreover, $H_1 \leq t, H_{t/\delta+1} \geq 0$. Thus

$$\Pr_{f \leftarrow [M]^m} [E_3] \leq \eta_{t/\delta} \leq \delta/t \sum_{i \in [t/\delta]} \eta_i \leq \frac{(1/\sqrt{\delta}) \cdot (H_{t/\delta+1} - H_1)}{t/\delta} \leq \sqrt{\delta}.$$

4.3 Proof of Theorem 4.6

Algorithm 2 in $\mathbb{A}$ guarantees that $\mathcal{U}(\{f \in \text{Cube}(p^*) : (f, y) \in S\}) < 1 - 3\sqrt{\delta}$ for all $y \in Y$. We can find a partition $Y = Y_P \sqcup Y_Q$ such that

$$\max \left\{ \mathcal{U} \left(\bigcup_{p \in L_{p^*}^P} \text{Cube}(p \cup p^*) \right), \mathcal{U} \left(\bigcup_{q \in L_{p^*}^Q} \text{Cube}(q \cup p^*) \right) \right\} \leq 1 - 3\sqrt{\delta},$$

where $L_{p^*}^P := \{\ell \in L_{p^*} : \exists y \in Y_P, \text{Cube}(\ell \cup p^*) \times \{y\} \subseteq S\}$ and $L_{p^*}^Q$ is defined analogously. Denote $\Gamma := L_{p^*}^P \times L_{p^*}^Q$. Algorithm 2 in $\mathbb{A}$ guarantees that $\mathcal{U}(\text{Uniq}(\text{Cube}(p^*))) \geq 1 - \sqrt{\delta}/2$. We can then conclude that

$$\begin{aligned} \mathcal{U}^{\otimes 2} \left(\bigcup_{(p,q) \in \Gamma} \text{Cube}(p \cup p^*) \times \text{Cube}(q \cup p^*) \right) &\geq \\ \mathcal{U} \left(\bigcup_{p \in L_{p^*}^P} \text{Cube}(p \cup p^*) \right) \cdot \mathcal{U} \left(\bigcup_{q \in L_{p^*}^Q} \text{Cube}(q \cup p^*) \right) &\geq \\ (1 - 3\sqrt{\delta}) \cdot (5/2)\sqrt{\delta}, \end{aligned}$$

where the last inequality holds since the sum of the multipliers on the left-hand side is $\mathcal{U}(\text{Uniq}(\text{Cube}(p^*))) \geq 1 - \sqrt{\delta}/2$ and the larger one of them is at most $1 - 3\sqrt{\delta}$. By applying Theorem 3.14, we have

$$\begin{aligned} \mathcal{U}^{\otimes 2} \left(\bigcup_{(p,q) \in \Gamma; p \sim^d q} C_p \times C_q \right) &\leq \mathcal{U}^{\otimes 2} \left(\bigcup_{(p,q) \in \Gamma; p \sim^d q} \text{Cube}(p \cup p^*) \times \text{Cube}(q \cup p^*) \right) \\ (\text{by Theorem 3.14}) &\leq \mathcal{U} \left(\bigcup_{(p,q) \in \Gamma; p \sim^d q} \text{Cube}(p \cup p^*) \cap \text{Cube}(q \cup p^*) \right) \\ &\leq 1 - \mathcal{U}(\text{Uniq}(\text{Cube}(p^*))) \\ &\leq \sqrt{\delta}/2. \end{aligned}$$

With $\alpha := \mathcal{U}^{\otimes 2} \left(\bigcup_{(p,q) \in \Gamma; p \sim^d q} C_p \times C_q \right)$ we then have

$$\begin{aligned} \alpha &= \mathcal{U}^{\otimes 2} \left(\bigcup_{(p,q) \in \Gamma} \text{Cube}(p \cup p^*) \times \text{Cube}(q \cup p^*) \right) - \mathcal{U}^{\otimes 2} \left(\bigcup_{(p,q) \in \Gamma; p \not\sim^d q} C_p \times C_q \right) \\ &\geq 5/2(1 - 3\sqrt{\delta})\sqrt{\delta} - \sqrt{\delta}/2 = 2\sqrt{\delta} - 7.5\delta \geq (3/2)\sqrt{\delta}, \end{aligned}$$

where the last inequality requires $\delta \leq 1/225$ which we have by the assumption. Equivalently,

$$\alpha = \sum_{p \in L_{p^*}^P} \mathcal{U}(C_p) \cdot \mathcal{U} \left(\bigcup_{q \in L_{p^*}^Q; q \sim^d p} C_q \right) = \sum_{p \in L_{p^*}^P} \mathcal{U}(C_p) F_{p^*}(p) \geq \frac{3}{2}\sqrt{\delta}.$$

Since $\sum_{p \in L_{p^*}^P} \mathcal{U}(C_p) \leq 1$, by averaging argument, there exists $\hat{\ell} \in L_{p^*}^P$ such that

$$\mathcal{U}(\mathbf{F}_{p^*}(\hat{\ell})) \geq \frac{3}{2}\sqrt{\delta}.$$

5 VDFs with computational uniqueness

We discuss VDFs with computational uniqueness in the ROM.

Theorem 5.1. *Suppose $\text{VDF} = (\text{Setup}, \text{Verify}, \text{Eval})$ is a VDF in the ROM with completeness error α . Fix $\lambda \in \mathbb{N}$. Let $q_{\text{Setup}}, q_{\text{Eval}}$ and q_{Verify} denote the query complexity of Setup, Eval and Verify, respectively. Then, for every $r_{\text{Adv}} > 2q_{\text{Verify}}$, there exists $\epsilon \geq 0$ such that VDF does not satisfy either $(q_{\text{Setup}} + r_{\text{Adv}}, q_{\text{Setup}} + r_{\text{Adv}} \cdot q_{\text{Eval}}, \gamma)$ -sequentiality for every $\gamma < 1 - \frac{2r_{\text{Adv}}}{r_{\text{Adv}} - 2q_{\text{Verify}}} \cdot \epsilon - \alpha$ or $(q_{\text{Adv}}, \epsilon)$ -uniqueness for $q_{\text{Adv}} = O(q_{\text{Verify}} \cdot q_{\text{Eval}})$.*

We show in Section A that Theorem 5.1 is tight by constructing a VDF in the ROM with statistical uniqueness and weaker sequentiality.

According to Section 3.4, it suffices to prove the following theorem:

Theorem 5.2. *Let $S \subseteq [M]^m \times Y$ be a search problem, determined by nondeterministic verifiers $\mathbb{V}$ of query complexity at most t . Let $\mathbb{D}: [M]^m \rightarrow Y \times \Pi$ be an algorithm of query complexity T such that $\Pr_{f \leftarrow [M]^m} [\mathbb{V}_{\mathbb{D}(f)}(f) = 1] \geq 1 - \alpha$. Then for every $t' > 2t$ there is some $\epsilon = \epsilon(t') \geq 0$ such that either*

1. *there exists a t' -round adversary $\mathbb{A}: [M]^m \rightarrow Y$ of query complexity $t'T$ such that*

$$\Pr_{f \leftarrow [M]^m} [\mathbb{A}(f) = \mathbb{D}^Y(f)] \geq 1 - \frac{2t'}{t' - 2t} \epsilon - \alpha; \text{ or}$$

2. *there exists an adversary $\mathbb{B}: [M]^m \rightarrow Y \times \Pi$ of query complexity $O(t'T)$ such that*

$$\Pr_{f \leftarrow [M]^m} [\mathbb{B}^Y(f) \neq \mathbb{D}^Y(f) \wedge \mathbb{V}_{\mathbb{B}(f)}(f) = 1] \geq \epsilon,$$

where $\mathbb{D}^Y(f)$ (resp. $\mathbb{B}^Y(f)$) is the Y -component of $\mathbb{D}(f)$ (resp. $\mathbb{B}(f)$).

Proof of Theorem 5.1 by Theorem 5.2. We devise two adversaries: one for breaking sequentiality, and the other for breaking computational uniqueness as follows: First, both adversaries run Setup to locate the leaf ℓ_f and sample $x \leftarrow \mathcal{X}$. Then each adversary executes the corresponding algorithm described in Theorem 5.2 for the search problem $S_{\ell_f, x}$. It follows that for every $r_{\text{Adv}} > 2q_{\text{Verify}}$, there is some $\epsilon = \epsilon(r_{\text{Adv}}) \geq 0$ (by averaging over all the search problems' individual ϵ) and either

1. *there exists a $(q_{\text{Setup}} + r_{\text{Adv}})$ -round $(r_{\text{Adv}} \cdot q_{\text{Eval}} + q_{\text{Setup}})$ -query adversary Adv such that*

$$\Pr \left[y = \text{Eval}^f(\text{pp}, x) \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{Setup}^f(1^\lambda, q_{\text{Eval}}) \\ x \leftarrow \mathcal{X} \\ y \leftarrow \text{Adv}^f(\text{pp}, x) \end{array} \right] \geq 1 - \frac{2r_{\text{Adv}}}{r_{\text{Adv}} - 2q_{\text{Verify}}} \cdot \epsilon - \alpha; \text{ or} \quad (1)$$

2. *there exists an adversary Adv of query complexity $O(r_{\text{Adv}} \cdot q_{\text{Eval}} + q_{\text{Setup}})$ such that*

$$\Pr \left[\begin{array}{l} y \neq \text{Eval}^f(\text{pp}, x) \\ \wedge \text{Verify}^f(\text{pp}, x, y, \pi) = 1 \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{O}(\lambda) \\ \text{pp} \leftarrow \text{Setup}^f(1^\lambda, q_{\text{Eval}}) \\ x \leftarrow \mathcal{X} \\ (y, \pi) \leftarrow \text{Adv}^f(\text{pp}, x) \end{array} \right] \geq \epsilon. \quad (2)$$

Taking for example $r_{\text{Adv}} = 3q_{\text{Verify}}$, whatever ϵ is, either (1) is non-negligible, or (2) is non-negligible. $\square$

5.1 The sequentiality breaker

We construct the adversary $\mathbb{A}$ below.

Algorithm 4 Adversary $\mathbb{A}$, the sequentiality breaker.

Input: $f \in [M]^m$

Output: $y \in Y \cup \{\perp\}$

- 1: $p^* := \star^m$.
 - 2: $K := \emptyset$.
 - 3: **for** $r \in [t']$ **do**
 - 4: Uniformly sample $f^* \in [M]^m$ consistent with p^* .
 - 5: Let ℓ be the unique leaf of $\mathbb{D}$ such that $\text{Cube}(\ell)$ contains f^* .
 - 6: $K := K \uplus \{\text{the solution associated with } \ell\}$.
 - 7: For every $j \in \text{dom}(\ell)$ such that $p^*[j] = \star$, query f at j and update $p^*[j]$ to be the query outcome.
 - 8: **return** the majority of solutions in K if it exists; $\perp$ otherwise.
-

Each iteration of $\mathbb{A}$ is a single round of queries, so $\mathbb{A}$ has t' rounds and makes at most T queries in each round, thus making at most $t'T$ queries in total.

To prove the correctness, we will first go through the execution of $\mathbb{A}$ and introduce some useful notations.

Let $\bar{F} := \{f : \mathbb{V}_{\mathbb{D}(f)}(f) = 1\}$ denote the set of functions computed correctly by $\mathbb{D}$. Recall that in each iteration, we choose some leaf ℓ of $\mathbb{D}$ according to some distribution conditioned on the current partial assignment p^* . Let y denote the solution associated with ℓ . For any input $f \in \bar{F}$, let $\ell_{\mathbb{V},f}$ denote the unique leaf of $\mathbb{V}_{\mathbb{D}(f)}$ such that $\text{Cube}(\ell_{\mathbb{V},f})$ contains f . We classify the iterations into three types according to f, p^*, ℓ :

1. $\mathbb{D}^Y(f) \neq y$ and $\text{Cube}(p^* \cup \ell) \cap \text{Cube}(p^* \cup \ell_{\mathbb{V},f}) = \emptyset$.
2. $\mathbb{D}^Y(f) \neq y$ and $\text{Cube}(p^* \cup \ell) \cap \text{Cube}(p^* \cup \ell_{\mathbb{V},f}) \neq \emptyset$.
3. $\mathbb{D}^Y(f) = y$.

Let $\mathbf{S}_{r,f}^{(1)}$ (resp. $\mathbf{S}_{r,f}^{(2)}$) be random indicator variables, which equals 1 if and only if the r -th iteration is the first type (resp. second type) for input f .

Intuitively, if both the first and the second type of iteration occur with low probability then we can prove $\mathbb{A}(f) = \mathbb{D}^Y(f)$ with high probability by simple Markov's inequality. Now assume that $\Pr_{f,r}[\mathbf{f} \in \bar{F} \wedge \mathbf{S}_{r,f}^{(2)} = 1]$ is negligible where r is uniformly sampled from $[t']$. We will prove $\Pr_r[\mathbf{S}_{r,f}^{(1)} = 1]$ is bounded for every $f \in \bar{F}$, which in turn implies $\mathbb{A}$ succeeds in simulating $\mathbb{D}$ with high probability. In Section 5.2 we show that there exists an adversary breaking the computational uniqueness condition if this assumption is false.

Lemma 5.3. *Let $\epsilon := \Pr_{f,r}[\mathbf{f} \in \bar{F} \wedge \mathbf{S}_{r,f}^{(2)} = 1]$. Then $\Pr_f[\mathbb{A}(\mathbf{f}) \neq \mathbb{D}^Y(\mathbf{f})] \leq \frac{2t'}{t'-2t}\epsilon + \alpha$.*

Proof. We first prove that $\sum_{r=1}^{t'} \mathbf{S}_{r,f}^{(1)} \leq t$ with probability 1 for every $f \in \bar{F}$. Consider the r -th iteration, if $\mathbf{S}_{r,f}^{(1)} = 1$, that is, $\text{Cube}(p^* \cup \ell) \cap \text{Cube}(p^* \cup \ell_{\mathbb{V},f}) = \emptyset$, then there exists some index $i \in \text{dom}(\ell) \cap \text{dom}(\ell_{\mathbb{V},f})$ such that $\ell[i] \neq \ell_{\mathbb{V},f}[i]$. The algorithm then queries $f[i]$ in this iteration. Thus, i will not be the inconsistent index in the later iterations. Since $|\text{dom}(\ell_{\mathbb{V},f})| \leq t$, we deduce that for every f , there can be at most t iterations such that $\text{Cube}(p \cup \ell) \cap \text{Cube}(p \cup \ell_{\mathbb{V},f}) = \emptyset$. Hence $\sum_{1 \leq r \leq t'} \mathbf{S}_{r,f}^{(1)} \leq t$ with probability 1.

Now let us combine the bound for $\sum_{r=1}^{t'} S_{r,f}^{(1)}$ with the assumption that $\Pr_{f,r}[f \in \bar{F} \wedge S_{r,f}^{(2)} = 1]$ is small. Let $\epsilon' := \frac{2t'}{t'-2t}\epsilon$. By Markov's inequality, for all but on average (over the internal randomness of $\mathbb{A}$) $(\epsilon' + \alpha)$ -fraction of $f \in [M]^m$ (recall $\alpha = 1 - \mathcal{U}_{[M]^m}(\bar{F})$), we have $f \in \bar{F}$ and $\sum_{r=1}^{t'} S_{r,f}^{(2)} < t\epsilon/\epsilon' = t'/2 - t$. For those f , $\sum_{r=1}^{t'} S_{r,f}^{(1)} + S_{r,f}^{(2)} < t'/2$. Thus the majority of recorded solutions are exactly $\mathbb{D}^Y(f)$. We conclude that the algorithm succeeds with probability at least $1 - \epsilon' - \alpha$. $\square$

5.2 The uniqueness breaker

Lemma 5.4. *Let $S_{r,f}^{(2)}$ be defined as in the last subsection and $\epsilon := \Pr_{f,r}[f \in \bar{F} \wedge S_{r,f}^{(2)} = 1]$. Then there exists an adversary $\mathbb{B}: [M]^m \rightarrow (Y \times \Pi) \cup \{\perp\}$ making $O(t'T)$ queries such that*

$$\Pr_{f \leftarrow [M]^m} [\mathbb{B}^Y(f) \neq \mathbb{D}^Y(f) \wedge \mathbb{V}_{\mathbb{B}(f)}(f) = 1] \geq \epsilon.$$

Proof. We construct the adversary $\mathbb{B}$ in Algorithm 5.

Algorithm 5 Adversary $\mathbb{B}$, the uniqueness breaker.

Input: $f \in [M]^m$

Output: $z \in (Y \times \Pi) \cup \{\perp\}$

- 1: Run $\mathbb{D}(f)$, let I be the set of indices queried during the execution.
 - 2: $p^* := \star^m$.
 - 3: **for** $r \in [t']$ **do**
 - 4: Uniformly sample $p' \leftarrow [M]^{I \setminus \text{dom}(p^*)}$.
 - 5: $f' := f_{(I \setminus \text{dom}(p^*)) \rightarrow p'}$.
 - 6: $(y, \pi) := \mathbb{D}(f')$.
 - 7: **if** $y \neq \mathbb{D}^Y(f) \wedge \mathbb{V}_{y,\pi}(f') = 1$ **then return** (y, π) .
 - 8: Uniformly sample $f^* \in [M]^m$ consistent with p^* .
 - 9: Let ℓ be the unique leaf of $\mathbb{D}$ such that $\text{Cube}(\ell)$ contains f^* .
 - 10: For every $j \in \text{dom}(\ell)$ such that $p^*[j] = \star$, query f at j and update $p^*[j]$ to be the query outcome.
 - 11: **return** $\perp$
-

Through the execution of $\mathbb{B}$, we can define $\mathcal{D}_{\mathbb{B}}$ as the following joint distribution of $(r \in [t'], p^* \in ([M] \cup \{\star\})^m, f \in [M]^m, f' \in [M]^m)$: Sample $f \leftarrow [M]^m, r \leftarrow [t']$ uniformly at random. Randomly simulate the for-loop in $\mathbb{B}$ on $f = f$ for $r - 1$ iterations. Let p^* denote the partial assignment at the start of the r -th iteration and f' denote the random function f' sampled in the r -th iteration of $\mathbb{B}$ (Line 5). See Figure 1 for visualization.

To prove the lemma, it suffices to show

$$\Pr_{(r, p^*, f, f') \leftarrow \mathcal{D}_{\mathbb{B}}} [\mathbb{D}^Y(f') \neq \mathbb{D}^Y(f) \wedge \mathbb{V}_{\mathbb{D}(f')}(f) = 1] \geq \epsilon. \quad (3)$$

To this end, we give an alternative view of $\mathcal{D}_{\mathbb{B}}$ based on the execution of $\mathbb{A}$.

First, we sample $f' \leftarrow [M]^m, r \leftarrow [t']$ uniformly at random. Then randomly simulate the for-loop in $\mathbb{A}$ on $f = f'$ for $r - 1$ iterations and let p^* denote the partial assignment p^* at the start of r -th iteration. Recall in the r -th iteration, we randomly choose some leaf ℓ of $\mathbb{D}$ conditioned on p^* , and denote the solution associated with ℓ by y . Let f denote the projection of $f = f'$ on $\text{Cube}(p^* \cup \ell)$. Formally, let $J := \text{dom}(\ell) \setminus \text{dom}(p^*)$ denote the set of indices fixed by ℓ but not by p^* and we can define $f := f_{J \rightarrow \ell_J}$.

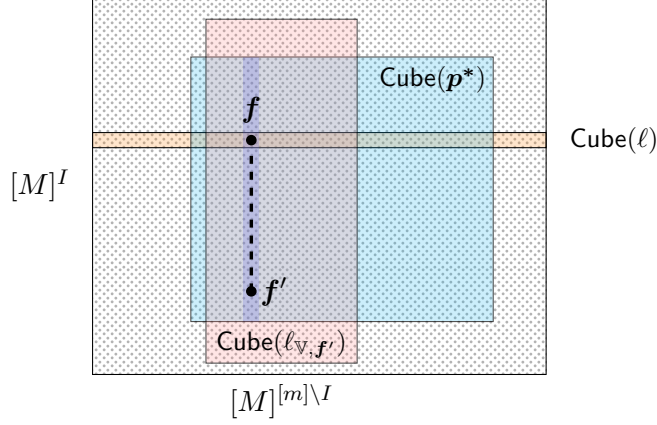


Figure 1: Distribution $\mathcal{D}_{\mathbb{B}}$.

Now observe that if $f' \in \bar{F}$ and $S_{r,f'}^{(2)} = 1$, then $\text{Cube}(p^* \cup \ell) \cap \text{Cube}(p^* \cup \ell_{V,f'}) \neq \emptyset$. Since $f \in \text{Cube}(\ell \cup p^*)$, f is consistent with $\ell_{V,f'}$ on I . Moreover, f equals f' on $[m] \setminus I$, and $\text{Cube}(\ell_{V,f'})$ includes f' , hence f is consistent with $\ell_{V,f'}$ on $[m] \setminus I$. We can deduce that $f \in \text{Cube}(\ell_{V,f'})$, which immediately implies $\mathbb{V}_{\mathbb{D}(f')}(f) = 1$. Note that we also have $\mathbb{D}^Y(f') = y \neq \mathbb{D}^Y(f)$ by the definition of $S_{r,f'}^{(2)} = 1$.

Finally, let $\mathcal{D}_{\mathbb{A}}$ denote the distribution of (r, p^*, f, f') according to the above sampling process. Since $f' \in \bar{F} \wedge S_{r,f'}^{(2)} = 1$ implies that $\mathbb{D}^Y(f') \neq \mathbb{D}^Y(f) \wedge \mathbb{V}_{\mathbb{D}(f')}(f) = 1$, we can deduce that

$$\Pr_{(r,p^*,f,f') \leftarrow \mathcal{D}_{\mathbb{A}}} [\mathbb{D}^Y(f') \neq \mathbb{D}^Y(f) \wedge \mathbb{V}_{\mathbb{D}(f')}(f) = 1] \geq \Pr_{r,f'} [f' \in \bar{F} \wedge S_{r,f'}^{(2)} = 1] = \epsilon.$$

Thus to prove (3), it suffices to show $\mathcal{D}_{\mathbb{B}} \equiv \mathcal{D}_{\mathbb{A}}$, that is, for every $r \in [t'], p^* \in ([M] \cup \{\star\})^m, f, f' \in [M]^m$,

$$\Pr_{\mathcal{D}_{\mathbb{A}}} [r = r, p^* = p^*, f = f, f' = f'] = \Pr_{\mathcal{D}_{\mathbb{B}}} [r = r, p^* = p^*, f = f, f' = f'].$$

Lemma 5.5. $\mathcal{D}_{\mathbb{A}} \equiv \mathcal{D}_{\mathbb{B}}$.

Proof. We need the following four statements.

Claim 5.6. For every $r \in [t']$, $\Pr_{\mathcal{D}_{\mathbb{A}}} [r = r] = \Pr_{\mathcal{D}_{\mathbb{B}}} [r = r]$.

Proof. Trivial since the marginal distributions of r are both uniform under $\mathbb{A}$ and $\mathbb{B}$. □

Claim 5.7. For every $r \in [t']$ and $p^* \in ([M] \cup \{\star\})^m$, $\Pr_{\mathcal{D}_{\mathbb{A}}} [p^* = p^* \mid r = r] = \Pr_{\mathcal{D}_{\mathbb{B}}} [p^* = p^* \mid r = r]$.

Proof. In both $\mathbb{A}$ and $\mathbb{B}$, p^* is the transcript of the query outcomes the following random process repeated for $r - 1$ times: Sample a uniformly random f^* consistent with the query outcome so far. Simulate $\mathbb{D}$ on f^* and query all the variables on the corresponding root-to-leaf path. □

Claim 5.8. For every $r \in [t'], p^* \in ([M] \cup \{\star\})^m$ such that $\Pr_{\mathcal{D}_{\mathbb{A}}} [p^* = p^* \mid r = r] > 0$, and every $f \in [M]^m$, $\Pr_{\mathcal{D}_{\mathbb{A}}} [f = f \mid r = r, p^* = p^*] = \Pr_{\mathcal{D}_{\mathbb{B}}} [f = f \mid r = r, p^* = p^*]$.

Proof. Conditioned on $\mathbf{r} = r, \mathbf{p}^* = p^*$, it is easy to see that $\mathbf{f}'$ is uniformly distributed over $\text{Cube}(p^*)$ under $\mathcal{D}_{\mathbb{A}}$ and $\mathbf{f}$ is uniformly distributed over $\text{Cube}(p^*)$ under $\mathcal{D}_{\mathbb{B}}$ by Bayes' rule. It suffices to show that $\mathbf{f}$ is also uniformly distributed over $\text{Cube}(p^*)$ under $\mathcal{D}_{\mathbb{A}}$.

Recall in the r -th round of $\mathbb{A}$, we choose some leaf ℓ of $\mathbb{D}$, and ℓ is chosen with probability $|\text{Cube}(p^* \cup \ell)|/|\text{Cube}(p^*)|$. Note that $\mathbf{f} \in \text{Cube}(p^* \cup \ell)$, we only need to prove $\mathbf{f}$ is uniformly distributed over $\text{Cube}(p^* \cup \ell)$ conditioned on ℓ is chosen. This is obvious since $\mathbf{f}'$ is uniformly distributed over $\text{Cube}(p^*)$, and by definition, $\mathbf{f}$ is the projection of $\mathbf{f}'$ on $\text{Cube}(p^* \cup \ell)$.

To conclude, $\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f} = f \mid \mathbf{r} = r, \mathbf{p}^* = p^*] = \frac{|\text{Cube}(p^* \cup \ell)|}{|\text{Cube}(p^*)|} \cdot \frac{1}{|\text{Cube}(p^* \cup \ell)|} = \frac{1}{|\text{Cube}(p^*)|}$. $\square$

Claim 5.9. For every $r \in [t']$, $p^* \in ([M] \cup \{\star\})^m$ such that $\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{p}^* = p^* \mid \mathbf{r} = r] > 0$, and every $f \in \text{Cube}(p^*)$, $\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f} = f] = \Pr_{\mathcal{D}_{\mathbb{B}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f} = f]$.

Proof. Without loss of generality, we assume that $f' \in \text{Cube}(p^*)$, as otherwise, $\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f} = f] = \Pr_{\mathcal{D}_{\mathbb{B}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f} = f] = 0$.

By Bayes' rule,

$$\begin{aligned} & \Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f} = f] \\ &= \frac{\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*] \cdot \Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f} = f \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f}' = f']}{\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f} = f \mid \mathbf{r} = r', \mathbf{p}^* = p^*]} \\ &= \Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f} = f \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f}' = f']. \end{aligned}$$

where the second equality follows since $\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f} = f \mid \mathbf{r} = r', \mathbf{p}^* = p^*] = \Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*] = \frac{1}{|\text{Cube}(p^*)|}$. Let ℓ denote the unique leaf of $\mathbb{D}$ such that $f' \in \text{Cube}(\ell)$ and $I = \text{dom}(\ell) \setminus \text{dom}(p^*)$. Now observe that

$$\Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f} = f \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f}' = f'] = \begin{cases} |\text{Cube}(p^* \cup \ell)|/|\text{Cube}(p^*)| & f_{[m] \setminus I} = f'_{[m] \setminus I} \\ 0 & \text{otherwise} \end{cases}.$$

On the other hand, for $\mathcal{D}_{\mathbb{B}}$, given $\mathbf{r} = r, \mathbf{p}^* = p^*, \mathbf{f} = f, \mathbf{f}'$ uniformly from $\{f' : f'_{[m] \setminus I} = f_{[m] \setminus I}\}$. Thus $\Pr_{\mathcal{D}_{\mathbb{B}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f} = f] = \Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f} = f \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f}' = f'] = \Pr_{\mathcal{D}_{\mathbb{A}}}[\mathbf{f}' = f' \mid \mathbf{r} = r', \mathbf{p}^* = p^*, \mathbf{f} = f]$, as desired. $\square$

Finally, by combining the above four claims and applying the chain rule, we deduce that $\mathcal{D}_{\mathbb{A}} \equiv \mathcal{D}_{\mathbb{B}}$. $\square$

To summarize, our uniqueness breaker $\mathbb{B}$ satisfies that

$$\begin{aligned} & \Pr_{\mathbf{f} \leftarrow [M]^m} [\mathbb{B}^Y(\mathbf{f}) \neq \mathbb{D}^Y(\mathbf{f}) \wedge \mathbb{V}_{\mathbb{B}(\mathbf{f})}(\mathbf{f}) = 1] = \\ & \Pr_{(\mathbf{r}, \mathbf{p}^*, \mathbf{f}, \mathbf{f}') \leftarrow \mathcal{D}_{\mathbb{B}}} [\mathbb{D}^Y(\mathbf{f}') \neq \mathbb{D}^Y(\mathbf{f}) \wedge \mathbb{V}_{\mathbb{D}(\mathbf{f}')}(\mathbf{f}) = 1] = \\ & \Pr_{(\mathbf{r}, \mathbf{p}^*, \mathbf{f}, \mathbf{f}') \leftarrow \mathcal{D}_{\mathbb{A}}} [\mathbb{D}^Y(\mathbf{f}') \neq \mathbb{D}^Y(\mathbf{f}) \wedge \mathbb{V}_{\mathbb{D}(\mathbf{f}')}(\mathbf{f}) = 1] \geq \\ & \Pr_{\mathbf{r}, \mathbf{f}'} [\mathbf{f}' \in \bar{\mathbb{F}} \wedge \mathbf{S}_{\mathbf{r}, \mathbf{f}'}^{(2)} = 1] = \epsilon. \end{aligned}$$

$\square$

5.3 Computational efficiency of the breakers

In this section we explain how to efficiently implement our sequentiality breaker $\mathbb{A}$ and uniqueness breaker $\mathbb{B}$.

Lemma 5.10. *Suppose that Eval is computable in time t_{Eval} and Verify computable in time t_{Verify} . Then time complexity of the sequentiality adversary $\mathbb{A}$ (Algorithm 4) and the uniqueness adversary $\mathbb{B}$ (Algorithm 5) are both $\text{poly}(t_{\text{Verify}} \cdot t_{\text{Eval}})$.*

Theorem 5.10 follows directly by the following implementation of the breakers (Algorithm 6 and Algorithm 7).

Algorithm 6 Uniform version of the sequentiality breaker.

Input: $\text{pp}; x \in \mathcal{X}$; oracle access to $f: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$

Output: $z \in Y \cup \{\perp\}$

- 1: $K := \emptyset$.
 - 2: $p^* := \emptyset$. $\triangleright p^* \subseteq \{0, 1\}^* \times \{0, 1\}^\lambda$.
 - 3: Define function $\text{dom}(p) := \{x \in \{0, 1\}^* \mid \exists y \in \{0, 1\}^\lambda: (x, y) \in p\}$ returning the set of the first elements of a set of pairs.
 - 4: **for** $r \in [t']$ **do**
 - 5: Uniformly sample $f^*: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ consistent with p^* . $\triangleright$ See Theorem 5.11
 - 6: $(y, \pi) := \text{Eval}^{f^*}(\text{pp}, x)$;
 - 7: $K := K \uplus \{y\}$.
 - 8: Let $\ell \subseteq \{0, 1\}^* \times \{0, 1\}^\lambda$ be the set of query-answer pairs from the execution in Line 6.
 - 9: For every $z \in \text{dom}(\ell) \setminus \text{dom}(p^*)$ query $f(z)$ and update $p^* := p^* \cup (z, k)$ where $(z, k) \in \ell$.
 - 10: **return** majority of K if it exists an $\perp$ otherwise.
-

Algorithm 7 Uniform version of the uniqueness breaker.

Input: $\text{pp}; x \in \mathcal{X}$; oracle access to $f: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$

Output: $z \in (Y \times \Pi) \cup \{\perp\}$

- 1: $(y_0, \pi_0) := \text{Eval}^f(\text{pp}, x)$; let I be the set of random oracle queries made during the execution.
 - 2: $p^* := \emptyset$. $\triangleright p^* \subseteq \{0, 1\}^* \times \{0, 1\}^\lambda$.
 - 3: Define function $\text{dom}(p) := \{x \in \{0, 1\}^* \mid \exists y \in \{0, 1\}^\lambda: (x, y) \in p\}$ returning the set of the first elements of a set of pairs.
 - 4: **for** $r \in [t']$ **do**
 - 5: Uniformly sample $p' \leftarrow (\{0, 1\}^\lambda)^{I \setminus \text{dom}(p^*)}$.
 - 6: $f' := f_{(I \setminus \text{dom}(p^*)) \rightarrow p'}$. $\triangleright$ Here we only mean it symbolically, see Theorem 5.11 for details.
 - 7: $(y, \pi) := \text{Eval}^{f'}(\text{pp}, x)$.
 - 8: **if** $y \neq y_0 \wedge \mathbb{V}^{f'}(\text{pp}, y, \pi) = 1$ **then return** (y, π) .
 - 9: Uniformly sample $f^*: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ consistent with p^* . $\triangleright$ See Theorem 5.11
 - 10: Run $\text{Eval}^{f^*}(\text{pp}, x)$;
 - 11: Let $\ell \subseteq \{0, 1\}^* \times \{0, 1\}^\lambda$ be the set of query-answer pairs from the execution in Line 10.
 - 12: For every $z \in \text{dom}(\ell) \setminus \text{dom}(p^*)$ query $f(z)$ and update $p^* := p^* \cup (z, k)$ where $(z, k) \in \ell$.
 - 13: **return** $\perp$
-

Remark 5.11. When we write assignments to oracles, we mean those to be defined lazily. In particular, the oracle defined in Line 6 is evaluated as follows: when $f'(z)$ is queried we first check if $z \in I \setminus \text{dom}(p^*)$, if it is we return $p'(z)$, otherwise we query $f(z)$ and return the answer. The oracle defined in Line 9 is evaluated as follows: when $f^*(z)$ is queried we first check if $z \in \text{dom}(p^*)$, if it is we return the unique k such that $(z, k) \in p^*$, otherwise if z was queried before we return the previously returned value, otherwise we sample k from $\{0, 1\}^\lambda$ uniformly at random and return k .

Remark 5.12. It is clear that the sequentiality breaker $\mathbb{A}$ runs in time $\text{poly}(t_{\text{Verify}} \cdot t_{\text{Eval}})$. However, $\mathbb{A}$ is not parallelizable. If one can construct a sequentiality breaker that runs in parallel time smaller than t_{Eval} , it would contradict the construction in [EFKP20], which presents a VDF that satisfies computational uniqueness and sequentiality in the ROM, assuming the hardness of repeated squaring. Hence, only a polynomial improvement is possible in the time complexity in either of our breakers unless the RSW assumption [RSW96] fails.

Acknowledgments

We thank Eylon Yogev for detailed feedback on an earlier draft of this paper and for inspiring the definition for proof of work functions. We thank Alessandro Chiesa and Mika G  s for the helpful discussions. The authors are supported by the Swiss State Secretariat for Education, Research and Innovation (SERI) under contract number MB22.00026. Ziyi Guan is partially supported by the Ethereum Foundation.

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. 1st. USA: Cambridge University Press, 2009. ISBN: 0521424267.
- [AC23] Hamza Abusalah and Valerio Cini. “An Incremental PoSW for General Weight Distributions”. In: *Proceedings of the 42th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’23. 2023, pp. 282–311.
- [AFGK22] Hamza Abusalah, Georg Fuchsbauer, Peter Ga  i, and Karen Klein. “SNACKs: Leveraging Proofs of Sequential Work for Blockchain Light Clients”. In: *Proceedings of the 28th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’22. 2022, pp. 806–836.
- [AKKPW19] Hamza Abusalah, Chethan Kamath, Karen Klein, Krzysztof Pietrzak, and Michael Walter. “Reversible Proofs of Sequential Work”. In: *Proceedings of the 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’19. 2019, pp. 277–291.
- [Abu23] Hamza Abusalah. *SNACKs for Proof-of-Space Blockchains*. IACR Cryptology ePrint Archive, Report 2023/806. 2023.
- [BBBF18] Dan Boneh, Joseph Bonneau, Benedikt B  nz, and Ben Fisch. “Verifiable Delay Function”. In: *Proceedings of the 38th Annual International Cryptology Conference*. CRYPTO ’18. 2018.
- [BBF18] Dan Boneh, Benedikt B  nz, and Ben Fisch. *A Survey of Two Verifiable Delay Functions*. IACR Cryptology ePrint Archive, Report 2018/712. 2018.
- [BCG24] Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. *Relativized Succinct Arguments in the ROM Do Not Exist*. Cryptology ePrint Archive, Paper 2024/728. 2024. URL: <https://eprint.iacr.org/2024/728>.
- [BI87] Manuel Blum and Russell Impagliazzo. “Generic oracles and oracle classes”. In: *Proceedings of the 28th Annual Symposium on Foundations of Computer Science*. SFCS ’87. USA: IEEE Computer Society, 1987, 118–126. ISBN: 0818608072. DOI: 10.1109/SFCS.1987.30.
- [BK85] J. Van Den Berg and H. Kesten. “Inequalities with applications to percolation and reliability”. In: *Journal of Applied Probability* 22.3 (1985), 556–569. DOI: 10.2307/3213860.
- [BPR15] Nir Bitansky, Omer Paneth, and Alon Rosen. “On the Cryptographic Hardness of Finding a Nash Equilibrium”. In: *Proceedings of the 56th Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’15. 2015, pp. 1480–1498.
- [Bac02] Adam Back. *Hashcash: A Denial-of-Service Counter-Measure*. <http://www.hashcash.org/papers/hashcash.pdf>. Technical Memo. 2002.
- [Bit+22] Nir Bitansky, Arka Rai Choudhuri, Justin Holmgren, Chethan Kamath, Alex Lombardi, Omer Paneth, and Ron D. Rothblum. “PPAD is as Hard as LWE and Iterated Squaring”. In: *Proceedings of the 20th Theory of Cryptography Conference*. TCC ’22. 2022, pp. 593–622.
- [CP18] Bram Cohen and Krzysztof Pietrzak. “Simple Proofs of Sequential Work”. In: *Proceedings of the 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’18. 2018, pp. 451–467.

- [DGMV20] Nico Döttling, Sanjam Garg, Giulio Malavolta, and Prashant Vasudevan. “Tight verifiable delay functions”. In: *Proceedings of the 15th International Conference on Security and Cryptography for Networks*. SCN ’20. 2020, pp. 65–84.
- [DLM19] Nico Döttling, Russell W. F. Lai, and Giulio Malavolta. “Incremental Proofs of Sequential Work”. In: *Proceedings of the 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’19. 2019, pp. 292–323.
- [DN93] Cynthia Dwork and Moni Naor. “Pricing via Processing or Combatting Junk Mail”. In: *Advances in Cryptology — CRYPTO’ 92*. Ed. by Ernest F. Brickell. Springer Berlin Heidelberg, 1993, pp. 139–147.
- [EFKP20] Naomi Ephraim, Cody Freitag, Ilan Komargodski, and Rafael Pass. “Continuous Verifiable Delay Functions”. In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’20. 2020, pp. 125–154.
- [FMPS19] Luca De Feo, Simon Masson, Christophe Petit, and Antonio Sanso. “Verifiable Delay Functions from Supersingular Isogenies and Pairings”. In: *Proceedings of the 25th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’19. 2019, pp. 248–277.
- [GKL15] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. “The Bitcoin Backbone Protocol: Analysis and Applications”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*. Springer, 2015, pp. 281–310.
- [HHKK23] Charlotte Hoffmann, Pavel Hubáček, Chethan Kamath, and Tomáš Krňák. “(Verifiable) Delay Functions from Lucas Sequences”. In: *Proceedings of the 20th Theory of Cryptography Conference*. TCC ’23. 2023.
- [HN23] Mathias Hall-Andersen and Jesper Buus Nielsen. “On Valiant’s Conjecture: Impossibility of Incrementally Verifiable Computation from Random Oracles”. In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’23. 2023.
- [HY17] Pavel Hubáček and Eylon Yogev. “Hardness of Continuous Local Search: Query Complexity and Cryptographic Lower Bounds”. In: *Proceedings of the 28th Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA ’17. 2017, 1352–1371.
- [JB99] Ari Juels and John Brainard. “Client Puzzles: A Cryptographic Defense Against Denial of Service Attacks”. In: *Proceedings of the 1999 Network and Distributed System Security Symposium (NDSS ’99)*. 1999.
- [JJ99] Markus Jakobsson and Ari Juels. “Proofs of Work and Bread Pudding Protocols(Extended Abstract)”. In: *Secure Information Networks: Communications and Multimedia Security IFIP TC6/TC11 Joint Working Conference on Communications and Multimedia Security (CMS’99) September 20–21, 1999, Leuven, Belgium*. Springer US, 1999, pp. 258–272. ISBN: 978-0-387-35568-9. DOI: 10.1007/978-0-387-35568-9_18. URL: https://doi.org/10.1007/978-0-387-35568-9_18.
- [KSS11] Jeff Kahn, Michael Saks, and Clifford Smyth. “The Dual BKR Inequality and Rudich’s Conjecture”. In: *Combinatorics, Probability and Computing* 20.2 (2011), 257–266. DOI: 10.1017/S0963548310000465.
- [LMCB12] Frank Li, Prateek Mittal, Matthew Caesar, and Nikita Borisov. “SybilControl: practical sybil defense with computational puzzles”. In: *Proceedings of the Seventh ACM Workshop on Scalable Trusted Computing*. STC ’12. Raleigh, North Carolina, USA: Association for Computing Machinery, 2012, 67–78. ISBN: 9781450316620. DOI: 10.1145/2382536.2382548. URL: <https://doi.org/10.1145/2382536.2382548>.
- [LV20] Alex Lombardi and Vinod Vaikuntanathan. “Fiat-Shamir for Repeated Squaring with Applications to PPAD-Hardness and VDFs”. In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO ’20. 2020, pp. 632–651.

- [MMV11] Mohammad Mahmoody, Tal Moran, and Salil Vadhan. “Time-Lock Puzzles in the Random Oracle Model”. In: *Proceedings of the 31st Annual International Cryptology Conference*. CRYPTO ’11. 2011, pp. 39–50.
- [MMV13] Mohammad Mahmoody, Tal Moran, and Salil Vadhan. “Publicly Verifiable Proofs of Sequential Work”. In: *Proceedings of the 4th Innovations in Theoretical Computer Science Conference*. ITCS ’13. 2013, 373–388.
- [MSW20] Mohammad Mahmoody, Caleb Smith, and David J. Wu. “Can Verifiable Delay Functions Be Based on Random Oracles?” In: *Proceedings of the 47th International Colloquium on Automata, Languages and Programming*. ICALP ’20. 2020, 83:1–83:17.
- [Pap94] Christos H. Papadimitriou. *Computational Complexity*. Reading, MA, USA: Addison-Wesley, 1994.
- [Pie19] Krzysztof Pietrzak. “Simple Verifiable Delay Functions”. In: *Proceedings of the 10th Innovations in Theoretical Computer Science Conference*. ITCS ’19. 2019, 60:1–60:15.
- [RSS20] Lior Rotem, Gil Segev, and Ido Shahaf. “Generic-Group Delay Functions Require Hidden-Order Groups”. In: *Advances in Cryptology – EUROCRYPT 2020*. Ed. by Anne Canteaut and Yuval Ishai. Cham: Springer International Publishing, 2020, pp. 155–180. ISBN: 978-3-030-45727-3.
- [RSW96] R. L. Rivest, A. Shamir, and D. A. Wagner. *Time-Lock Puzzles and Timed-Release Crypto*. Tech. rep. 1996.
- [Rei00] David Reimer. “Proof of the Van den Berg–Kesten Conjecture”. In: *Combinatorics, Probability and Computing* 9.1 (2000), 27–32. DOI: 10.1017/S0963548399004113.
- [Rud88] Steven Rudich. *Limits on the Provable Consequences of One-way Functions*. Tech. rep. USA, 1988.
- [Smy02] Clifford Smyth. “Reimer’s inequality and tardos’ conjecture”. In: *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing*. STOC ’02. Montreal, Quebec, Canada: Association for Computing Machinery, 2002, 218–221. ISBN: 1581134959. DOI: 10.1145/509907.509942. URL: <https://doi.org/10.1145/509907.509942>.
- [Sta20] StarkWare Industries. *Presenting: VeeDo*. <https://medium.com/starkware/presenting-veedo-e4bbff77c7ae>. 2020.
- [Val08] Paul Valiant. “Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency”. In: *Proceedings of the 5th Theory of Cryptography Conference*. TCC ’08. 2008, pp. 1–18.
- [Wes19] Benjamin Wesolowski. “Efficient verifiable delay functions”. In: EUROCRYPT ’19 (2019), p. 623.

A Tightness of Theorem 5.1

Theorem 5.1 is essentially “tight” in terms of sequentiality: a VDF can be constructed in the ROM with statistical uniqueness and weaker sequentiality.

Lemma A.1. *Fix $\lambda \in \mathbb{N}$ and $T \in \mathbb{N}$. There exists a VDF = (Setup, Eval, Verify) in which $q_{\text{Setup}} = 0$, $q_{\text{Eval}} = T + 1$, and $q_{\text{Verify}} = O(1)$ that satisfies*

- perfect completeness,
- $(q_{\text{Adv}}, \epsilon)$ -uniqueness for unbounded q_{Adv} and $\epsilon = \text{negl}(\lambda)$, and
- $(r_{\text{Adv}}, q'_{\text{Adv}}, \gamma)$ -sequentiality for every $r_{\text{Adv}} \in \mathbb{N}$, $q'_{\text{Adv}} = 2^{\lambda(T/r_{\text{Adv}} - 1) - 1}$, and $\gamma \geq 1 - \epsilon/4$.

In Theorem 5.1, we have that sequentiality error γ is upper bounded by $1 - \frac{2r_{\text{Adv}}}{r_{\text{Adv}} - 2q_{\text{Verify}}} \cdot \epsilon - \alpha$, which is at most $1 - 2\epsilon - \alpha$. Therefore, Theorem A.1 complements Theorem 5.1 by arguing for the existence of VDFs with perfect completeness and relaxed sequentiality error $\gamma \geq 1 - \epsilon/4$.

To show Theorem A.1, it suffices to prove the following lemma:

Lemma A.2. For any security parameter λ , query complexity parameter $T \in \mathbb{N}^+$. Let $n = (M^T - 1)/(M - 1) + 1$. Then there is a search problem $S \subseteq [2^\lambda]^n \times [2]$ defined by verifiers $\mathbb{V}_1, \mathbb{V}_2$ and an algorithm $\mathbb{D}$ computing S which satisfies the following:

- (i) Both verifiers $\mathbb{V}_1, \mathbb{V}_2$ have query complexity $O(1)$. $\mathbb{D}$ has query complexity $T + 1$.
- (ii) Exactly $1/2^\lambda$ -fraction of inputs have alternative solutions, i.e. there exists $z \in Y$ such that $(f, z) \in S$ but $z \neq \mathbb{D}(f)$.
- (iii) For every r -round adversary $\mathbb{A}$ with query complexity at most $2^{\lambda(T/r-1)-1}$,

$$\Pr \left[\mathbb{A}(f) = \mathbb{D}(f) \mid f \leftarrow [2^\lambda]^n \right] \leq 1 - \frac{1}{2^{\lambda+2}}.$$

To construct the search problem in Theorem A.2, we define the following hard (on average) functions against parallel decision trees.

Definition A.3. Let $M > 0$ be even, $T \in \mathbb{N}^+$. For $n := (M^T - 1)/(M - 1)$, let $h_{M,T}: [M]^n \rightarrow \{0, 1\}$ be the sequential function whose computation can be defined as a complete depth- T decision tree, where different non-leaf nodes are labeled with different variables. The leaf nodes are labeled with the parity of the variable associated with their respective parent nodes so that any non-trivial subtree is balanced, namely, the subtree contains an equal number of 0-leaves and 1-leaves.

Lemma A.4. Any r -round algorithm computing $h_{M,T}$ with success probability $3/4$ over the uniformly random input has query complexity at least $M^{\lfloor (T-1)/r \rfloor} / 2$.

Proof. Fix $\ell = \lfloor (T-1)/r \rfloor$. We prove by induction on $R \in \mathbb{N}$ that the following alternative statement holds: Any R -round algorithm of query complexity $Q^* \leq M^\ell$ computing $h_{M,k\ell+1}$ has success probability at most $(1 + Q^*/M^\ell)/2$.

When $R = 0$, any 0-round algorithm cannot make any queries. Since $h_{M,1}$ is 0 on exactly half of the inputs, the algorithm must compute $h_{M,1}$ with success probability exactly $1/2$.

Now assume that the statement is true when $R = k - 1 \geq 0$. Then for $R = k$ and any k -round algorithm $\mathbb{A}_k$ of query complexity Q^* computing $h_{M,k\ell+1}$. Let $I_0 \subseteq [n(M, k\ell + 1)]$ of size $|I_0| = Q_0$ denote the set of indices queried in the first round.

Recall that there is a complete depth- T decision tree computing $h_{M,k\ell+1}$, whose nodes are labeled with different variables. Let $w_1, \dots, w_{M^\ell}$ be all the nodes on the ℓ -th level. Moreover, for any $1 \leq v \leq M^\ell$, let I_v be the set of indices of variables that appear in the subtree with root w_v , $g_v := h_{M,k\ell+1}|_{\text{Cube}(w_v)}$. That is, g_v is a function mapping from $\text{Cube}(w_v)$ to $\{0, 1\}$ where $g_v(f) = h_{M,k\ell+1}(f)$ for all $f \in \text{Cube}(w_v)$. Let $V := \{v : I_v \cap I_0 \neq \emptyset\}$. By the definition of $h_{M,k\ell+1}$, $I_1, \dots, I_{M^\ell}$ are pairwise disjoint, so $|V| \leq |I_0| = Q_0$.

For any $v \in [M^\ell] \setminus V$, since $\mathbb{A}_k$ does not query any variable in I_v in the first round, it performs exactly the same as some $k - 1$ -round $Q^* - Q_0$ -query algorithm computing g_v . It follows from the induction hypothesis and the fact that g_v is isomorphic to $h_{M,(k-1)\ell+1}$ that $\mathbb{A}_k$ computes g_v with success probability at most $(1 + (Q^* - Q_0)/M^\ell)/2$.

Then we can bound the probability that $\mathbb{A}_k$ computes $h_{M,k\ell+1}$:

$$\begin{aligned} & \Pr_{f \leftarrow [M]^m} [\mathbb{A}_k(f) = h_{M,k\ell+1}(f)] \\ &= \frac{1}{M^\ell} \left(\sum_{v \in V} \Pr_{f \leftarrow \text{Cube}(w_v)} [\mathbb{A}_k(f) = h_{M,k\ell+1}(f)] + \sum_{v \in [M^\ell] \setminus V} \Pr_{f \leftarrow \text{Cube}(w_v)} [\mathbb{A}_k(f) = h_{M,k\ell+1}(f)] \right) \end{aligned}$$

$$\begin{aligned}
&\leq \frac{1}{M^\ell} \left(|V| + (M^\ell - |V|)(1 + (Q^* - Q_0)/M^\ell)/2 \right) \\
&\leq \frac{1}{M^\ell} \left(Q_0 + (M^\ell - Q_0)(1 + (Q^* - Q_0)/M^\ell)/2 \right) \\
&\leq (1 + Q^*/M^\ell)/2.
\end{aligned}$$

Finally, by replacing Q^* with $M^\ell/2$ and observing that $t \geq r\ell + 1$, we obtain the desired claim. $\square$

Proof of Theorem A.2. The search problem is defined by two verifiers $V_1, V_2 : [2^\lambda]^n \rightarrow \{0, 1\}$: V_1 accepts all the inputs, and V_2 only accepts f such that $f_1 = 1$.

Now let us define $\mathbb{D}$. For the set of input inputs $\{f : f_1 \neq 1\}$, $\mathbb{D}$ simply outputs 1. For rest of the inputs, we embed the sequential function $h_{2^\lambda, T}$ in the subcube $\{f : f_1 = 1\}$. Specifically, we define

$$\mathbb{D}(f) := \begin{cases} 1 & f_1 \neq 1 \\ h_{2^\lambda, T}(f_{[n] \setminus \{1\}}) + 1 & f_1 = 1 \end{cases}.$$

It is clear that (i)(ii) hold. Note that any algorithm computing $\mathbb{D}$ with success probability at least $1 - 2^{\lambda+2}$ also computes $h_{2^\lambda, T}$ with success probability at least $3/4$. By Theorem A.4, (iii) holds. $\square$